인공지능

Artificial Intelligence

시계열 데이터와 순환 신경망

본 강의자료는 한빛아카데미 〈파이썬으로 만드는 인공지능〉을 내용을 기반으로 제작 및 보완되었음을 알립니다.

8.1 시계열 데이터의 이해

- 시계열 데이터는 시간 축을 따라 신호가 변하는 동적 데이터임
- 지금까지 배운 SVM, MLP, DNN, CNN 은 정적 데이터를 한꺼번에 입력 받으므로 시계열 데이터를 처리하는데 적합하지 않음
- 시계열 데이터를 고정 길이의 정적 데이터로 바꾸는 기법을 적용할 수도 있지만 정보 손실이 크고 변환 기법을 사람이 설계해야 하는 부담이 있음
  - 영상 정보를 1D로 가공하여 DNN 입력으로 사용할 수 있으나 정보 손실이 큰 이유와 동일

- 딥러닝에서는 시계열 특성을 활용하는 순환 신경망(RNN, Recurrent Neural Network) 또는 장단기메모리(LSTM, Long Short Term Memory)을 주로 사용함

8.1 시계열 데이터의 이해

- [그림 8-1]은 다양한 시계열 데이터의 예
  - (a) 심전도 신호를 분석하는 의료 인공지능을 만들어 더욱 정확하게 병을 진단할 수 있음
  - (b) 주식 시세를 잘 분석하는 인공지능 제품을 만들면 투자에 유리할 수 있음
  - (c) 정확률이 높은 음성 인식기를 만들면 경쟁력 있는 인공지능 제품을 만들 수 있음
  - (d) 유전자 분석하는 인공지능을 만들면 신약 개발에 활용할 수 있음



(a) 심전도 (b) 주식 시세

(c) 음성 인식 데이터 (d) 유전자 염기 서열

그림 8-1 다양한 시계열 데이터

8.1.1 시계열 데이터의 특성

- 시계열 데이터의 특성 * 각 특성에 대해서 다음 장에 있음
  - 요소의 순서가 중요
  - 샘플의 길이가 다름
  - 문맥 의존성을 지님
  - 계절성을 가지는 데이터가 많음

8.1.1 시계열 데이터의 특성

- **요소의 순서가 중요**
  - “세상에는 시계열 데이터가 참 많다” 라는 문장을 “데이터가 시계열 많다 참 세상에는” 라는 문장으로 바꾸면 의미가 크게 훼손됨
  - 시간 정보가 없는 데이터는 특징의 순서를 바꿔 표현해도 아무런 문제가 없음
    - 예를 들어, iris 데이터에서는 샘플이 〈꽃잎 길이, 꽃잎 너비, 꽃받침 길이, 꽃받침 너비〉라는 특징 벡터로 표현되는데, 벡터의 요소 즉 특징의 순서를 바꿔 〈꽃잎 너비, 꽃받침 너비, 꽃잎 길이, 꽃받침 길이〉로 표현해도 상관없음

- **샘플의 길이가 다름**
  - 인공지능 이란 단어를 짧게 발음하는 사람도 있고 인~공~지~능으로 길게 발음하는 사람도 있음
  - 심전도를 잴 때도 심각한 환자는 더욱 정밀하게 진단하기 위해 더 길게 측정하기도 함
  - 유전자 염기 서열에서도 잡음이 섞이거나 중간에 빠지는 염기가 발생해 측정할 때마다 길이가 조금씩 다를 수 있음

8.1.1 시계열 데이터의 특성

- **문맥 의존성을 지님**
  - "시계열은 앞에서 말한 바와 같이 데이터를 구성하는 요소의 순서, 즉 나타나는 순서가 중요하다는 특성이 있다" 라는 문장에서 주어인 "시계열은"과 서술어인 "특성이 있다"는 밀접한 연관성이 있으며, 이런 연관성을 문맥 의존성 (context dependency)라고 함
  - 시계열을 처리하는 기계 학습 모델은 문맥 의존성을 스스로 발견하고 표현하는 능력이 있어야 함
  - 예시 문장에서 주어와 서술어 사이에 단어 12개가 있으므로 주어와 서술어는 12만큼 떨어져 있으며, 이처럼 연관성이 깊은 요소가 멀리 떨어져서 나타나는 상황을 장기 문맥 의존성 (long-term context dependency)라고 함

- **계절성을 가지는 데이터가 많음, seasonality**
  - 상추 판매량은 야외 바비큐에 적절한 계절에 매출이 치솟는 경향이 있음
  - 수제 맥주 전문점은 금요일에 매출이 치솟는 경향이 있음
  - 미세먼지 수치나 항공권 판매량은 계절에 영향이 있음

8.1.1 시계열 데이터의 특성

- **시계열 데이터(동적데이터)의 정의**

$$\mathbf{x}=\left(\mathbf{a}^1\ \mathbf{a}^2\cdots\mathbf{a}^t\right) \qquad (8.1)$$

  - 데이터의 길이가 가변적이므로 d 대신 t 로 표기함
  - X 벡터의 요소가 벡터이므로 이를 고려하여 굵은 글씨체로 표기함 $\mathbf{a}^i$

- **시계열 데이터의 예**
  - 매일 기온, 습도, 미세먼지 농도를 기록한다면,
    첫째날은 $\mathbf{a}^1=(23.5,42,0.1)$, 둘째날은 $\mathbf{a}^2=(25.5,45,0.08)$ 로 표현 가능
  - 심전도의 경우 10개의 채널 값이 들어오고, 30초 분량의 심전도를 초당 20번 샘플링한다면
    t=600인 샘플 X가 생성되는데, x의 요소 a^i 는 10차원 벡터에 해당됨

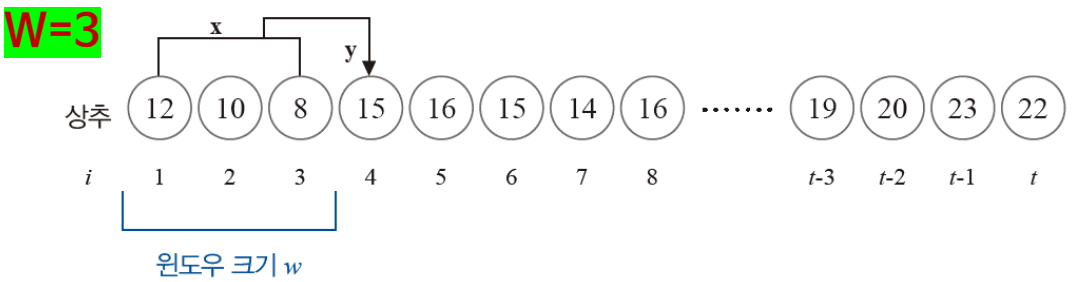
8.1.2 미래 예측을 위한 데이터 준비

- 순환 신경망은 주로 미래 예측에 사용됨 (즉“추정” 문제 해결)
  - 내일의 주가 예측
  - 내일의 날씨 예측
  - 내일의 판매량 예측

- 분류와 추정을 종종 예측이라 부르기 때문에, 이를 구분할 수 있어야 함
  - 새로운 숫자 패턴이 입력되면 어떤 숫자인지 “분류 classification”하는 일도 “예측 prediction”
  - 판매 시계열 데이터를 보고 미래의 판매량을 “추정 forecasting”하는 일도 “예측 prediction”

8.1.2 미래 예측을 위한 데이터 준비

- 시계열 패턴 하나로 미래를 예측하는 신경망을 어떻게 학습할 것인가?
  - 미래를 예측하는 문제에서는 아래와 같이 일정한 길이로 패턴을 잘라 여러 샘플을 만듦
  - 이때 이전 요소 몇 개를 학습에 사용할 것인가 의미하는 윈도우 크기 w 설정이 필요함



(a) 입력 데이터 샘플링

샘플	x	y
1	(12, 10, 8)	15
2	(10, 8, 15)	16
3	(8, 15, 16)	15
4	(15, 16, 15)	14
...	...	...
t-w	(19, 20, 23)	22

(b) 입력 패턴에서 생성한 샘플

- 위 그림은 〈이전 3일의 판매량을 보고 다음 날 판매량을 예측〉하기 위한 학습데이터 가공 예시
  - t=(1, 2, 3) 의 값을 보고 t=4 순간 값을 예측하고,

    t=(2, 3, 4) 의 값을 보고 t=5 순간 값을 예측하는 방식으로 학습 데이터를 준비함

8.1.2 미래 예측을 위한 데이터 준비

- 시계열 패턴 하나로 미래를 예측하는 신경망을 어떻게 학습할 것인가?

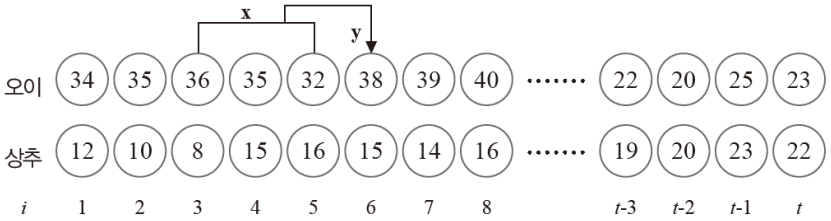
  - 〈이전 3일의 판매량을 보고 다음 날 판매량을 예측〉하기 위한 학습데이터 가공
    - t=(1, 2, 3) 의 값을 보고 t=4 순간 값을 예측하고,
      t=(2, 3, 4) 의 값을 보고 t=5 순간 값을 예측하는 방식으로 학습 데이터를 준비함

  - 〈이전 3일의 판매량을 보고 이틀 후 판매량을 예측〉하기 위한 학습데이터 가공
    - t=(1, 2, 3) 의 값을 보고 t=5 순간 값을 예측하고,
      t=(2, 3, 4) 의 값을 보고 t=6 순간 값을 예측하는 방식으로 학습 데이터를 준비함

  - 얼마나 먼 미래를 예측할 것인지 지정하는 요인을 수평선 계수 horizon factor 라고 하며, 위의 예시는 각각 수평선 계수 h=1 혹은 h=2의 문제임
  - 일반적으로, 수평선 계수가 클 수록 어려운 문제에 해당됨

8.1.2 미래 예측을 위한 데이터 준비

- 다중 시계열 패턴은 미래를 예측하는 신경망을 어떻게 학습할 것인가?
  - 품목이 2개 이상인 시계열 데이터를 동시에 처리해야 하는 다중 채널인 경우도 있음
  - 아래 그림은 다중 품목에 대한 〈이전 3일의 판매량을 보고 다음 날 판매량을 예측〉하기 위한 학습데이터 가공 예시임



(a) 입력 패턴

샘플	x	y
1	((34, 12), (35, 10), (36, 8))	(35, 15)
2	((35, 10), (36,8), (35, 15))	(32, 16)
3	((36, 8), (35, 15), (32, 16))	(38, 15)
4	((35, 15), (32, 16), (38, 15))	(39, 14)
...	...	...
t - *w*	((22, 19), (20, 20), (25, 23))	(23, 22)

(b) 입력 패턴에서 생성한 샘플

그림 8-3 미래 예측 문제에서 샘플 생성하기(다중 채널)

8.1.3 시계열 데이터 사례: 비트코인 가격

- 비트코인 가격 데이터 다뤄보기
  - 디지털 통화에 관한 뉴스 사이트 코인데스크에서 비트코인 데이터를 다운로드
    - 데이터 얻는 방법 교재와 같이 해보니, 달라졌음



(a) 코인데스크에서 데이터를 다운로드하기

B	C	D	E	F	G
Date	Closing Pr	24h Open	24h High	24h Low (USD)	
2019-02-28	3772.94	3796.64	3824.17	3666.52	
2019-03-01	3799.68	3773.44	3879.23	3753.8	
2019-03-02	3811.61	3799.37	3840.04	3788.92	
2019-03-03	3804.42	3806.69	3819.19	3759.41	
2019-03-04	3782.66	3807.85	3818.7	3766.24	
2019-03-05	3689.86	3783.36	3804.35	3663.48	
2019-03-06	3832.08	3701.05	3866.72	3688.7	
2019-03-07	3848.96	3832.59	3881.97	3802.52	
2019-03-08	3859.84	3848.96	3890.75	3827.67	
2019-03-09	3828.37	3859.8	3918	3778.52	
2019-03-10	3898.87	3841.89	3948.88	3832.14	
2019-03-11	3899.66	3916.76	3921.93	3865.92	
2019-03-12	3851.25	3899.46	3913.46	3819.93	

(b) 비트코인 가격이 저장된 데이터 프레임

그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

**다운로드 사이트의 화면이나 메뉴가 바뀌는 경우는 빈번하여, 책과 다른 화면이 나올 수 있음*

8.1.3 시계열 데이터 사례: 비트코인 가격

- 비트코인 가격 데이터 다뤄보기
  - (교재에서 제공하는) 파일을 열어보면, 아래의 그림과 같이 5개의 열이 있음
    - [Date, Closing Price, Open Price, High Price, Low Price] 로 구성 있음

[8]:

date	closing price	open price	high price	low price
2019- 02- 28	3816.6	3814.6	3883.7	3783.3
2019- 03- 01	3821.9	3816.7	3855.8	3816.4
2019- 03- 02	3823.1	3821.9	3843.2	3783.6
2019- 03- 03	3809.5	3823.2	3836.6	3789.7
2019- 03- 04	3715.9	3809.7	3828.4	3681.8

    - 시계열 데이터 역시 csv로 담겨 있으므로, pandas 라이브러리로 데이터를 읽고, matplotlib 라이브러리로 시각화를 진행해 보겠음

8.1.3 시계열 데이터 사례: 비트코인 가격

- **프로그램 8-1 (a) 비트코인 가격 데이터 읽기**

[18]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

coindesk_data_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/traindata.csv",header=0)
coindesk_data_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/testdata.csv",header=0)
```

```
coindesk_data_train.head(5)
```

[19…

	date	closing price	open price	high price	low price
0	2019-02-28	3816.6	3814.6	3883.7	3783.3
1	2019-03-01	3821.9	3816.7	3855.8	3816.4
2	2019-03-02	3823.1	3821.9	3843.2	3783.6
3	2019-03-03	3809.5	3823.2	3836.6	3789.7
4	2019-03-04	3715.9	3809.7	3828.4	3681.8

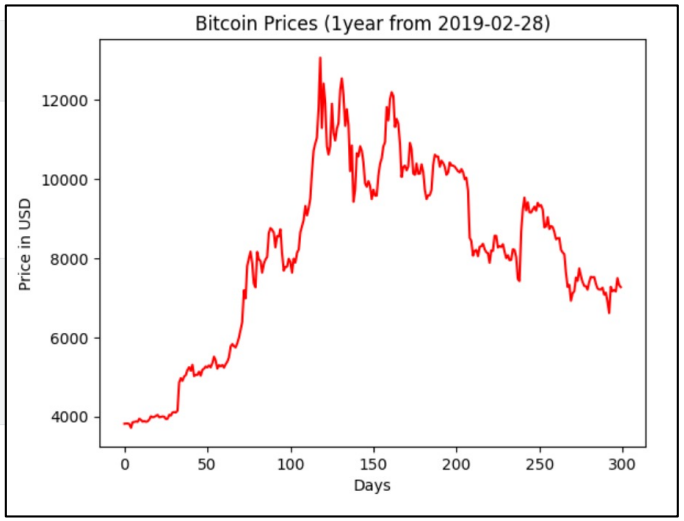
← 종가, 시가, 최고가, 최저가

+ Code + Markdown

[20]:
```
seq = coindesk_data_train[['closing price']].to_numpy()
print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])
```

```
데이터 길이: 300
앞쪽 5개 값: [[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]]
```

[22]:
```
# 그래프로 데이터 확인
plt.plot(seq,color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p1-codingnote

8.1.3 시계열 데이터 사례: 비트코인 가격

- **프로그램 8-1 (a) 비트코인 가격 데이터 읽기**

[18]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

coindesk_data_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/traindata.csv",header=0)
coindesk_data_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/testdata.csv",header=0)
```

```
coindesk_data_train.head(5)
```

[19…]

	date	closing price	open price	high price	low price
0	2019-02-28	3816.6	3814.6	3883.7	3783.3
1	2019-03-01	3821.9	3816.7	3855.8	3816.4
2	2019-03-02	3823.1	3821.9	3843.2	3783.6
3	2019-03-03	3809.5	3823.2	3836.6	3789.7
4	2019-03-04	3715.9	3809.7	3828.4	3681.8

+ Code + Markdown

[20]:
```
seq = coindesk_data_train[['closing price']].to_numpy()
print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])
```

```
데이터 길이: 300
앞쪽 5개 값: [[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]]
```

[22]:
```
# 그래프로 데이터 확인
plt.plot(seq,color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```

NOTE [[ ]]와 [ ]의 차이

[프로그램 8-1(a)]에서 08행의 coindesk_data[['Closing Price (USD)']] 구문에 [[…]]을 사용했다. 08행을 coindesk_data['Closing Price (USD)']로 바꾸어 […] 방식을 사용하면 무슨 차이가 있을까? 간단한 예를 들면, [[…]] 방식은 [[3] [5] [7] [6]]과 같이 표현되고 […] 방식은 [3 5 7 6]으로 표현된다. [프로그램 8-1(a)]는 매일 형성되는 네 가지 주식 시세인 종가, 시가, 고가, 저가 중에서 종가만 취하여 실험을 하기 때문에 벡터의 벡터인 [[…]]와 벡터인 […]의 차이가 두드러지지 않는다. 네 값을 모두 사용하는 다중 채널의 경우는 반드시 벡터의 벡터인 [[…]] 표현을 사용해야 한다. 종가만 사용하는 프로그램에서 [[…]] 표현으로 코딩해두면 다중 채널로 확장이 쉬워지기 때문에 더 좋은 프로그램이 된다. 다중 채널로 확장하는 작업은 [프로그램 8-3]에서 다룬다.

[[]] vs []의 차이!

```
seq2 = coindesk_data_train['closing price'].to_numpy()
```

```
seq2
```

```
array([ 3816.6,  3821.9,  3823.1,  3809.5,  3715.9,  3857.2,  3863. ,
        3875.1,  3865.9,  3944.3,  3915.2,  3870.3,  3886. ,  3865.1,
        3879. ,  3924.3,  4006.4,  3981.5,  3990.2,  4017. ,  4041.2,
        3982.2,  3990.4,  4002.5,  3994.7,  3937. ,  3942.8,  4041.7,
        4025.6,  4102.2,  4111.8,  4102.3,  4145.1,  4859.3,  4968.7,
        4902.4,  5010.2,  5046.2,  5173.6,  5245.2,  5158.4,  5307.8,
        5022.6,  5054.2,  5051.8,  5134.8,  5032.3,  5180.9,  5208.3,
        5264.7,  5241. ,  5290.2,  5243.5,  5346.7,  5511.6,  5415.6,
        5209.1,  5298.3,  5265.9,  5302.3,  5235. ,  5320.8,  5384.2,
        5493.8,  5766.8,  5830.9,  5774.9,  5745.1,  5849.5,  5990.3,
        6191.5,  6386. ,  7190.3,  6984.8,  7806. ,  7994.6,  8164.6,
        7871.8,  7359.5,  7262.6,  8157.2,  7965.3,  7930.3,  7635.7,
        7852.1,  7970.1,  8027.4,  8630.2,  8760.1,  8716.3,  8647.8
```

```
seq2 = coindesk_data_train[['closing price','open price']].to_numpy()
```

```
seq2
```

```
array([[ 3816.6,  3814.6],
       [ 3821.9,  3816.7],
       [ 3823.1,  3821.9],
       [ 3809.5,  3823.2],
       [ 3715.9,  3809.7],
       [ 3857.2,  3715.9],
       [ 3863. ,  3857.2],
       [ 3875.1,  3863.1],
       [ 3865.9,  3875.1],
       [ 3944.3,  3865.6],
       [ 3915.2,  3944.4],
       [ 3870.3,  3915.2],
       [ 3886. ,  3870.3],
       [ 3865.1,  3885.9],
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p1-codingnote

8.1.3 시계열 데이터 사례: 비트코인 가격

- **프로그램 8-2(b) 1년치 비트코인 가격 데이터를 윈도우로 자르기** 시계열 데이터 전처리
  - 이제 시계열 데이터를 윈도우 단위로 나누고, 수평계수를 반영하여 샘플링 해야함

```
seq = coindesk_data_train[['closing price']].to_numpy()
print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])
```

```
데이터 길이: 300
앞쪽 5개 값: [[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]]
```

+ Code + Markdown

[18]:

```
def seq2dataset(seq, window, horizon):
    X = []; Y=[]
    for i in range(len(seq)-(window+horizon)+1):
        x=seq[i:(i+window)]
        y=(seq[i+window+horizon-1])
        X.append(x)
        Y.append(y)
    return np.array(X),np.array(Y)
```

[19]:

```
# W = 이전데이터 수, h = 수평선계수
w=7; h=1;

X, Y = seq2dataset(seq,w,h)

print(X[0],Y[0])
```

```
[[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]
 [3857.2]
 [3863. ]] [3875.1]
```

date	closing price	open price	high price	low price
2019-02-28	3816.6	3814.6	3883.7	3783.3
2019-03-01	3821.9	3816.7	3855.8	3816.4
2019-03-02	3823.1	3821.9	3843.2	3783.6
2019-03-03	3809.5	3823.2	3836.6	3789.7
2019-03-04	3715.9	3809.7	3828.4	3681.8
2019-03-05	3857.2	3715.9	3873.2	3705.7
2019-03-06	3863.0	3857.2	3887.3	3816.7
2019-03-07	3875.1	3863.1	3907.4	3847.9
2019-03-08	3865.9	3875.1	3929.0	3810.7
2019-03-09	3944.3	3865.6	3964.0	3859.7

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p2-codingnote

8.2 순환 신경망

- 시계열 데이터를 처리하는 신경망을 설계할 때는 시간에 따라 값이 하나씩 순차적으로 들어온다는 사실을 고려해야 함

- 컨볼루션 신경망(CNN)과 깊은 다층 퍼셉트론(MLP)에서는 데이터가 맨 왼쪽에 있는 입력층으로 한꺼번에 들어온다는 사실을 감안하면, 순환 신경망(RNN)은 이전 신경망과 구조 측면에서 본질적으로 달라야 할 것임
- 순환 신경망(RNN)은 다행스럽게도 다층 퍼셉트론(MLP)을 약간 고쳐서 사용 가능함
  - [주의] 개념 설명을 위해 도입된 것임. 완전 일치하지는 않음

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
  - [그림8-5(a)]는 [그림4-13]을 다시 그린 것으로, **순환 신경망(RNN, Recurrent Neural Network)**과 비교하기 위해 왼쪽에서 오른쪽으로 데이터가 흐르던 방식을 아래에서 위로 흐르게 90도 회전해 그렸음
  - 또한 U^1과 U^2로 표기하던 가중치 집합을 U와 V로 바꾸어 표기함

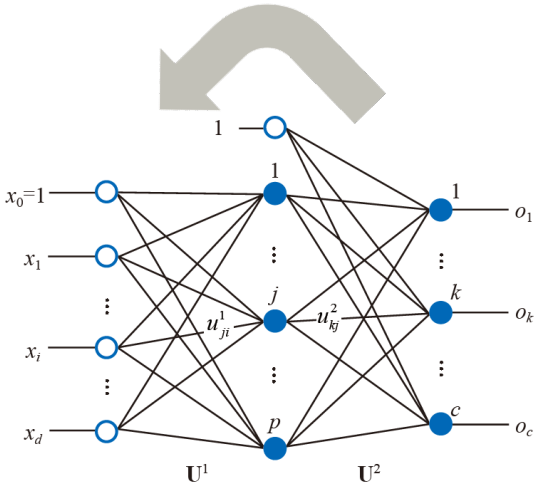
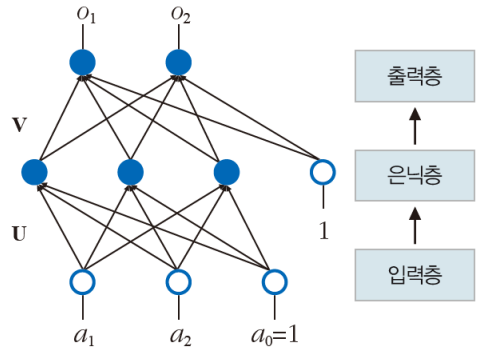
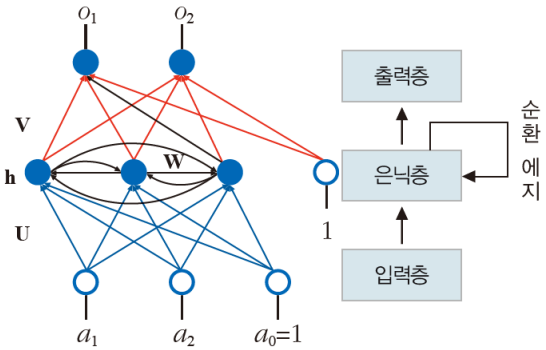


그림 4-13 다층 퍼셉트론의 구조



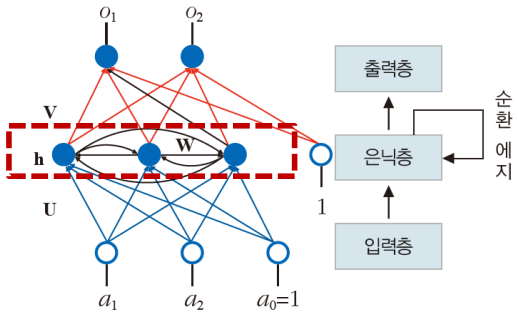
(a) 다층 퍼셉트론([그림 4-13])



(b) 순환 신경망([그림 4-13]에 순환 에지 추가)

그림 8-5 다층 퍼셉트론과 순환 신경망의 비교

8.2.1 구조와 동작, 학습 알고리즘



(b) 순환 신경망([그림 4-13]에 순환 에지 추가)

- 시계열 데이터를 처리하는 신경망
  - 순환 신경망은 다층 퍼셉트론과 한 가지만 다름
  - 순환 신경망에는 은닉층에 있는 노드끼리 에지로 연결되어 있음
  - 이 에지는 은닉층 내에서 정보를 순환하므로 **순환 에지(recurrent edge)**라고 함
  - "순환 에지"라는 단순한 아이디어로 〈다층 퍼셉트론(MLP)〉이 시계열 데이터 처리에 적합한 〈순환 신경망(RNN)〉으로 변신한 것임

  - 순환 신경망은 순환 에지를 통해 시간 정보를 입력하고 가변 길이를 다루고 문맥 정보를 처리하는 능력을 갖출 수 있음
    - 시간 정보를 입력 : 데이터가 시계열 순서데로 입력
    - 가변 길이를 다룸 : 입력 길이 만큼 RNN 반복 (입력 길이를 미리 알 필요 없음)
    - 문맥 정보 처리 : 이전 은닉 상태를 현재 은닉 상태에 반영

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
  - 순환 에지는 어떻게 시간 정보를 처리할 수 있을까?
  - 아래 [그림 8-6]은 [그림8-5(b)]의 순환 신경망을 단지 펼친 것

  - [그림8-6]의 신경망에는 1이라는 순간, 2라는 순간, …, t라는 순간이 있음
  - i 순간에는 입력층 a^i가 입력되고, 은닉층의 상태가 h^i로 변하고 출력층에서 o^i가 출력됨
  - [그림 8-5(b)]의 a_0, a_1, a_2가 [그림 8-6]의 a^i를 구성

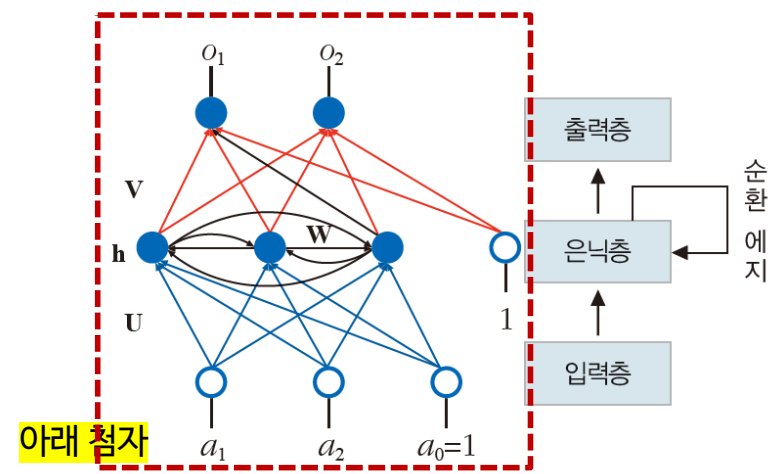


그림 8-5 (b) 순환 신경망([그림 4-13]에 순환 에지 추가)

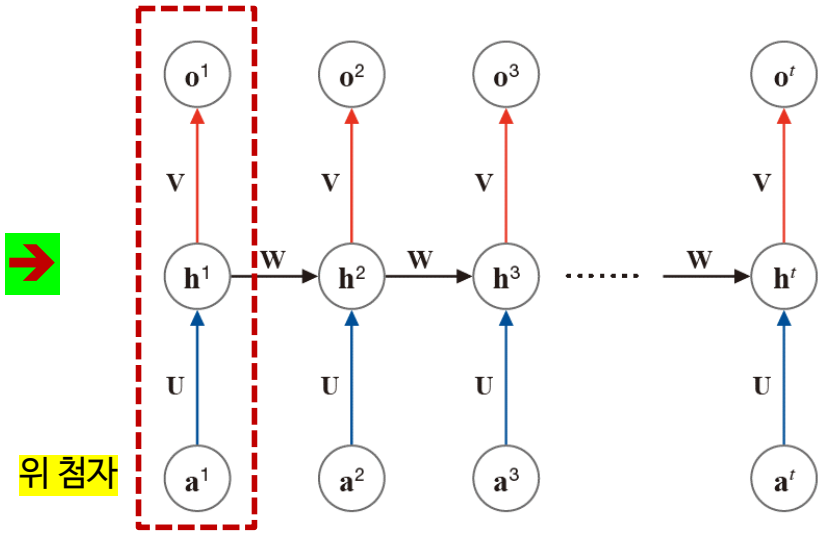


그림 8-6 펼친 순환 신경망

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
  - 순환 신경망이 다층 퍼셉트론과 두드러지게 다른 점은 가중치 집합임
    - 다층 퍼셉트론 {U, V} vs. 순환 신경망 {U, V, W}
    - 가중치 U, V, W를 보면 시간을 나타내는 첨자가 없으며, 이는 순환 신경망이 모든 순간 가중치를 공유한다는 것을 의미함

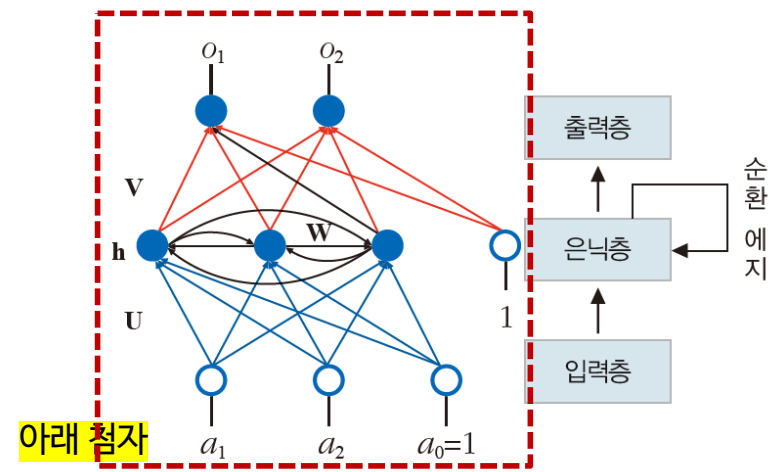


그림 8-5 (b) 순환 신경망([그림 4-13]에 순환 에지 추가)

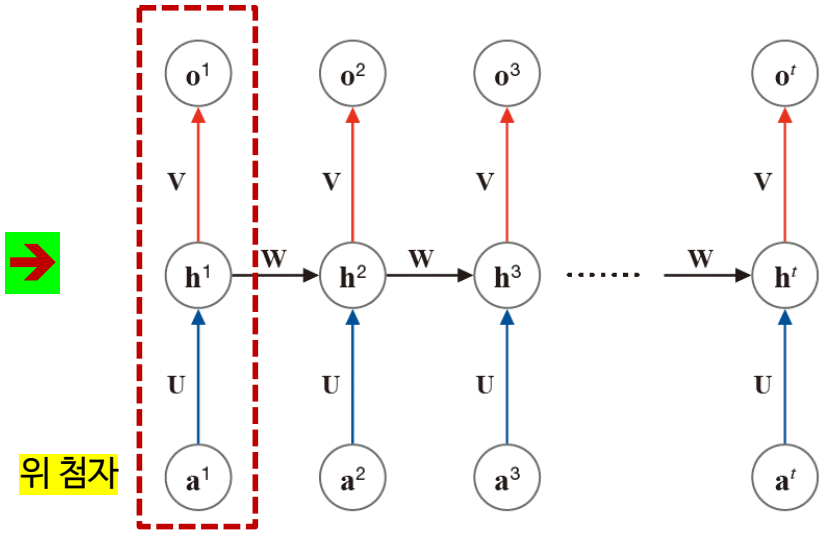


그림 8-6 펼친 순환 신경망

8.2.1 구조와 동작, 학습 알고리즘

- 순환 신경망의 동작
  - i 순간의 a^i는 가중치 **U**를 통해 은닉층의 상태 h^i에 영향을 미치고, h^i는 가중치 **V**를 통해 출력값 o^i에 영향을 미침. h^{i-1}는 가중치 **W**를 통해 h^i에 영향을 미침
  - 은닉층에서 일어나는 계산

$$\mathbf{h}^i=\tau_1\left(\mathbf{W}\mathbf{h}^{i-1}+\mathbf{U}\mathbf{a}^i\right) \qquad (8.2)$$

  - 출력층에서 일어나는 계산

$$\mathbf{o}^i=\tau_2\left(\mathbf{V}\mathbf{h}^i\right) \qquad (8.3)$$

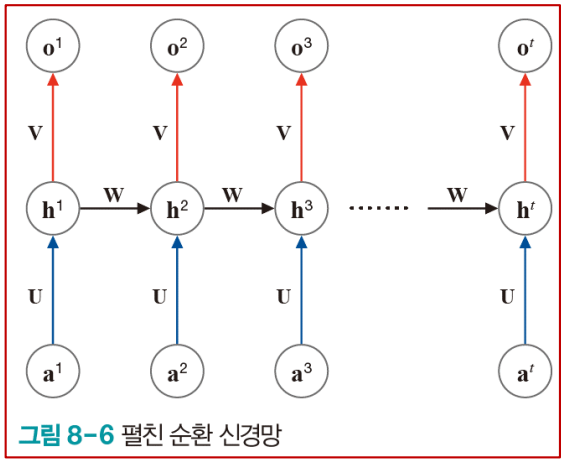
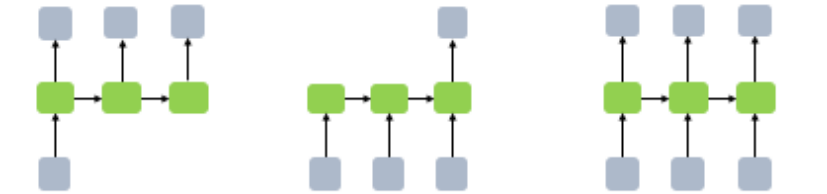


그림 8-6 펼친 순환 신경망

  - 식 (8.2)에서 $\mathbf{W}h^{i-1}$ 항을 제외하면 다층 퍼셉트론과 동일
    - 순환 신경망은 이 항을 통해 이전 순간의 은닉층 상태 h^{i-1}를 현재 순간의 은닉층 상태 h^i로 전달하여 시간성을 처리함
  - 순환 신경망의 학습 알고리즘을 BPTT(Back-propagation Through Time)라고 하며, 기존 CNN 신경망들의 back-propagation과는 다름

8.2.1 구조와 동작, 학습 알고리즘

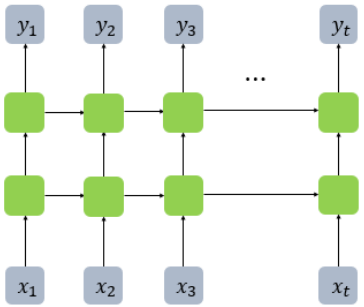
- RNN은 입력과 출력의 길이를 다르게 설계할 수 있으므로 다양한 용도로 사용 가능함



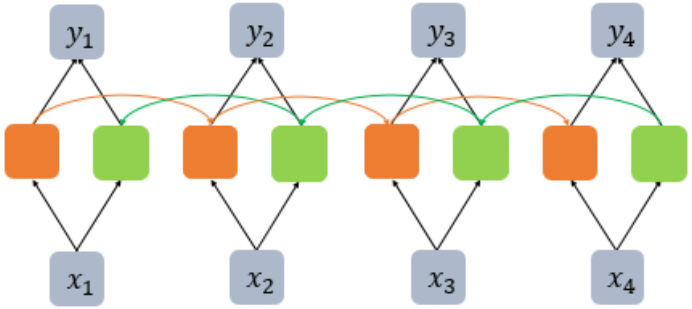
  - 일 대 다(one-to-many) : 하나의 이미지 입력에 대해 사진의 제목을 출력하는 이미지 캡셔닝 작업에 사용
  - 다 대 일(many-to-one) : 단어의 시퀀스에 대해서 하나의 출력을 하는 스팸 분류에 사용
  - 다 대 다(many-to-many) : 입력 문장으로부터 대답 문장을 출력하는 챗봇, 번역기 등

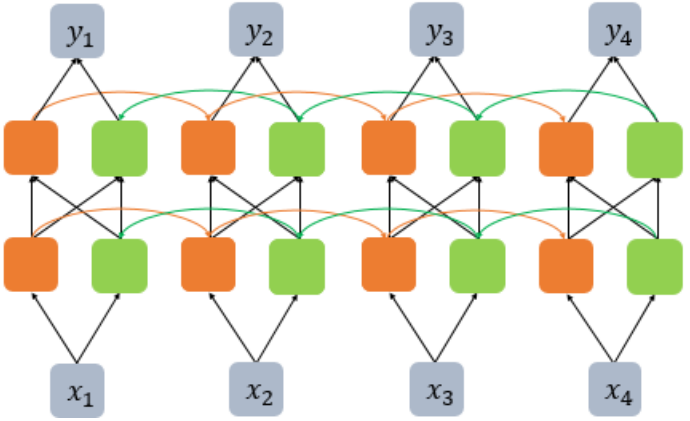
8.2.1 구조와 동작, 학습 알고리즘

- RNN이 다수의 은닉층을 가지는 경우를 깊은 순환 신경망(Deep RNN) 이라고 함



- 양방향 순환 신경망(Bidirectional RNN)은 시점 t에서의 출력 값을 예측할 때, 이전 시점의 입력 뿐만 아니라, 이후 시점의 입력 또한 예측에 기여할 수 있다는 아이디어에서 기반함





8.2.1 구조와 동작, 학습 알고리즘

- RNN 파이토치 실습
  - 단층 단방향 기본 RNN (은닉층 1개, Basic RNN)
    - 일 대 다(one-to-many)
    - 다 대 일(many-to-one)
  - 다층 RNN (은닉층 2개 이상, Deep RNN)
  - 양방향 RNN (Bidirectional)

```
class torch.nn.RNN(input_size, hidden_size, num_layers=1,
nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0,
bidirectional=False, device=None, dtype=None)                [source]
```

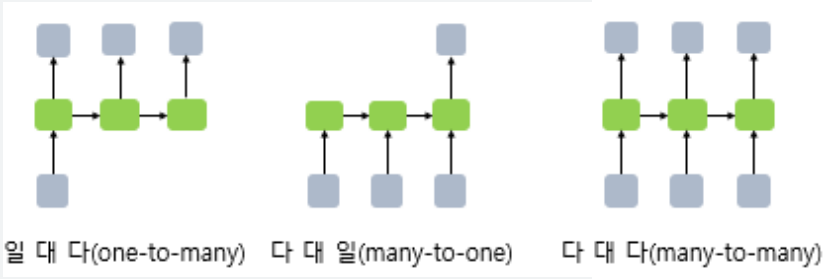
https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p4-codingnote

8.2.1 구조와 동작, 학습 알고리즘

- 다 대 일(many-to-one)

```python
class ManyToOneRNN(nn.Module):
    """
    예: 문장(토큰 시퀀스)을 입력 받아 감정 3가지(긍정/부정/중립)로 분류한다고 가정
    입력:  (batch, seq_len, input_size)
    출력:  (batch, num_classes)
    """
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        out, h_n = self.rnn(x)      # out: (batch, seq_len, hidden_size)
        last_hidden = out[:, -1]    # 마지막 타임스텝만 사용 (batch, hidden_size)
        logits = self.fc(last_hidden)  # (batch, num_classes)
        return logits
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p4-codingnote

8.2.1 구조와 동작, 학습 알고리즘

- 다 대 일(many-to-one)
  - 단층 RNN - 다 대 일 구조 : num_layers = 1
  - 깊은 RNN - 다 대 일 구조 : num_layers = 2

```
# ---- 사용 예시 ----
batch_size   = 4
seq_len      = 10   # 문장 길이 10 토큰
input_size   = 16   # 각 토큰 임베딩 차원
hidden_size  = 32
num_layers   = 1
num_classes  = 3    # 감정 3종류

model = ManyToOneRNN(input_size, hidden_size, num_layers, num_classes)

x = torch.randn(batch_size, seq_len, input_size)
logits = model(x)

print("Many-to-One 출력 shape:", logits.shape)  # (4, 3)

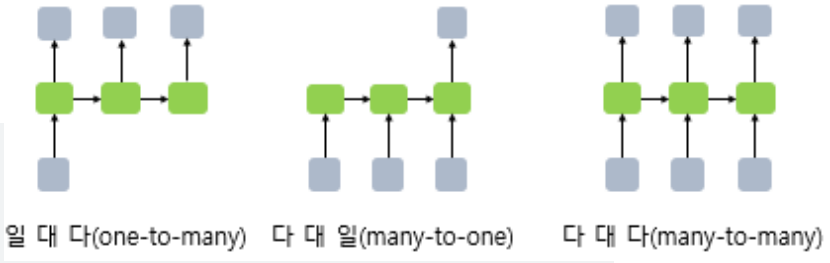
```

```
Many-to-One 출력 shape: torch.Size([4, 3])
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p4-codingnote

8.2.1 구조와 동작, 학습 알고리즘

- 다 대 다(many-to-many)




```python
class ManyToManyRNN(nn.Module):
    """
    예: 입력 시퀀스 각 토큰마다 태그를 붙이는 POS 태깅/NER 등
    입력:  (batch, seq_len, input_size)
    출력:  (batch, seq_len, num_tags)
    """
    def __init__(self, input_size, hidden_size, num_layers, num_tags):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, num_tags)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        out, h_n = self.rnn(x)          # out: (batch, seq_len, hidden_size)
        tag_logits = self.fc(out)       # (batch, seq_len, num_tags)
        return tag_logits
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p4-codingnote

8.2.1 구조와 동작, 학습 알고리즘

- 다 대 다(many-to-many)
  - 단층 RNN – 다 대 일 구조 : num_layers = 1
  - 깊은 RNN – 다 대 일 구조 : num_layers = 2

```
# ---- 사용 예시 ----
batch_size   = 4
seq_len      = 10
input_size   = 16
hidden_size  = 32
num_layers   = 1
num_tags     = 5   # 예: 태그 5종류

model = ManyToManyRNN(input_size, hidden_size, num_layers, num_tags)

x = torch.randn(batch_size, seq_len, input_size)
tag_logits = model(x)

print("Many-to-Many 출력 shape:", tag_logits.shape)  # (4, 10, 5)

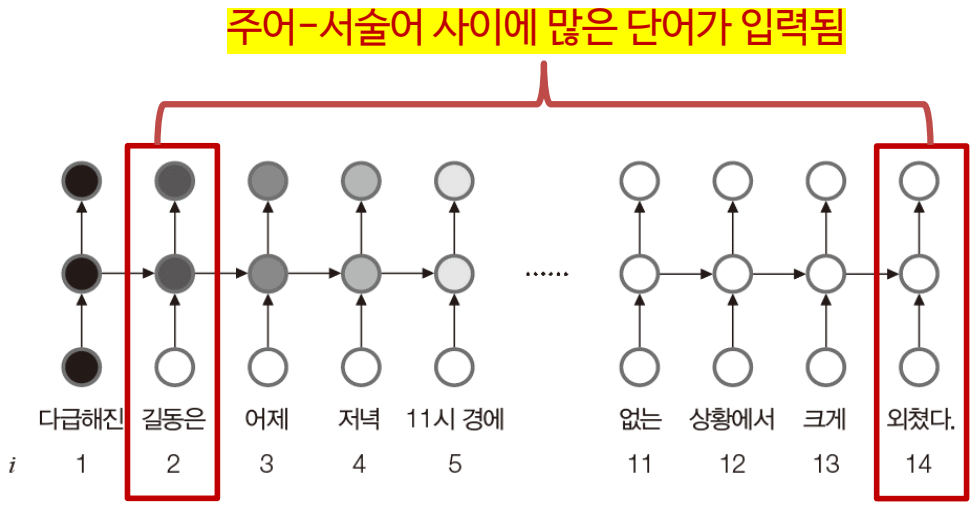
```

```
Many-to-Many 출력 shape: torch.Size([4, 10, 5])
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-p4-codingnote

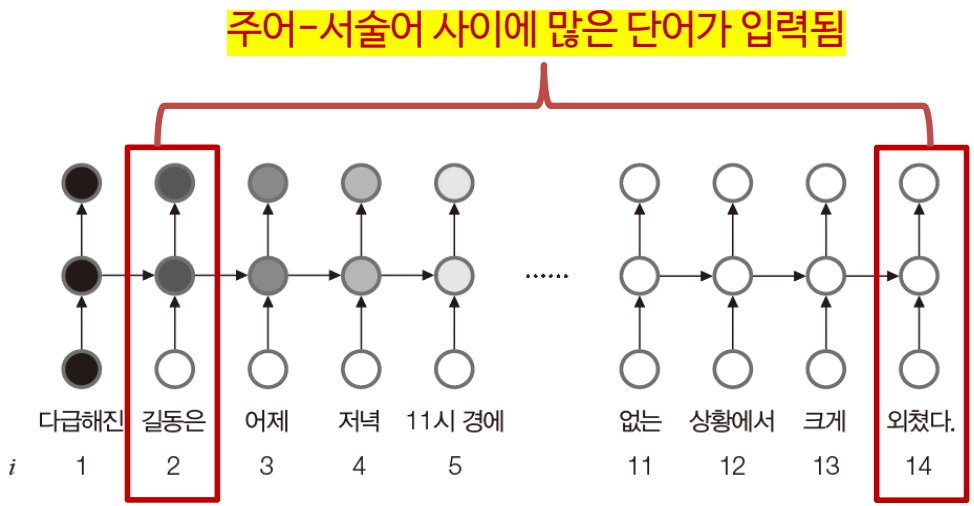
8.2.2 선별 기억력을 갖춘 LSTM

- 순환 신경망(RNN)은 이전 순간의 은닉층 상태를 다음 순간으로 넘기는 기능을 통해 과거를 기억하는 능력이 있으나, 장기 문맥 의존성을 제대로 처리하지 못하는 단점이 있음
- **장기 문맥 의존성 (long-term context dependency)**은 떨어져 있는 두 요소가 밀접한 상호작용을 하는 성질을 말하며, 많은 시계열 데이터가 장기 문맥 의존성을 가짐
  - 아래 예시) 주어-서술어는 밀접한 관련성이 있음. 이런 관련성을 알아야 번역과 같은 일에 성공적 수행 가능
- 장기 문맥 의존성을 제대로 처리하지 못하는 이유는 〈계속 들어오는 입력의 영향〉으로 기억력이 감퇴하는 것이며, 이는 시간이 지남에 따라 기억이 점점 희미해지는 사람 뇌에 비유할 수 있음



8.2.2 선별 기억력을 갖춘 LSTM

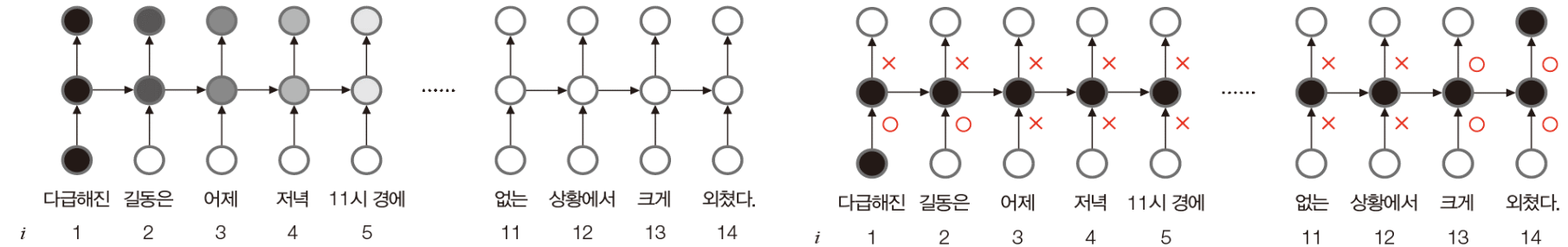
- 사람은 **선별 기억 능력** selective memory 이 있어 인상적인 일은 아주 오래 전 기억도 간직함
- **LSTM** Long Short-Term Memory 은 순환 신경망에 **선별 기억 능력** selective memory 을 추가한 신경망임
- 언어 번역기나 챗봇 같은 인공지능 제품을 만들 때 LSTM을 활용함
- LSTM은 게이트gate 라는 개념을 통해 선별 기억 능력을 확보함



8.2.2 선별 기억력을 갖춘 LSTM

- [그림 8-7(a)] 순환 신경망(RNN)이 시간이 지남에 따라 기억이 희미해지는 현상을 설명함
- 순간2에 입력된 "길동은" 이라는 단어는 순간 3,4,5에는 기억에 남아 있지만, 이후에는 거의 사라지는 현상이 나타남

- [그림 8-7(b)] 순환 신경망에 "게이트gate"를 추가한 LSTM 임
- O는 열린 상태의 게이트로서 신호를 전송하고, X는 닫힌 상태의 게이트로서 신호를 차단함
- 단, 게이트는 열고 닫는 두 가지 상태만 있는 것이 아니라, 여닫는 정보를 조절할 수 있도록 게이트 0~1 사이의 실수 값을 가짐
- 이 원리에 따라 LSTM은 게이트를 이용하여 선별적으로 기억하는 능력을 갖춤



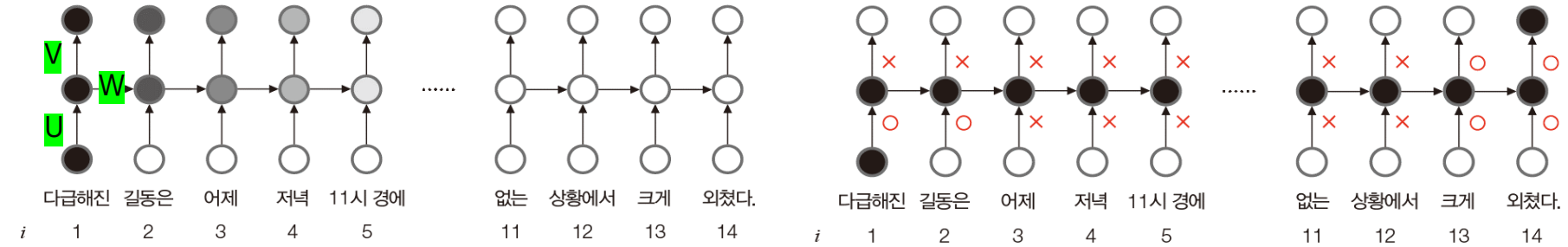
(a) 선별 기억 능력이 없는 순환 신경망

(b) 선별 기억 능력을 지닌 LSTM

그림 8-7 순환 신경망과 LSTM

8.2.2 선별 기억력을 갖춘 LSTM

- [그림8-7(b)]는 LSTM을 아주 간단화한 설명임
- 게이트의 여닫는 정도는 LSTM의 가중치로 표현되며, 가중치 값은 학습으로 알아냄
- LSTM의 가중치는 순환 신경망의 {U, V, W}에 4개를 추가하여 {U, U^i, U^o, W, W^i, W^o, V}에 해당되며, i는 입력 게이트, o는 출력 게이트를 의미함



(a) 선별 기억 능력이 없는 순환 신경망

(b) 선별 기억 능력을 지닌 LSTM

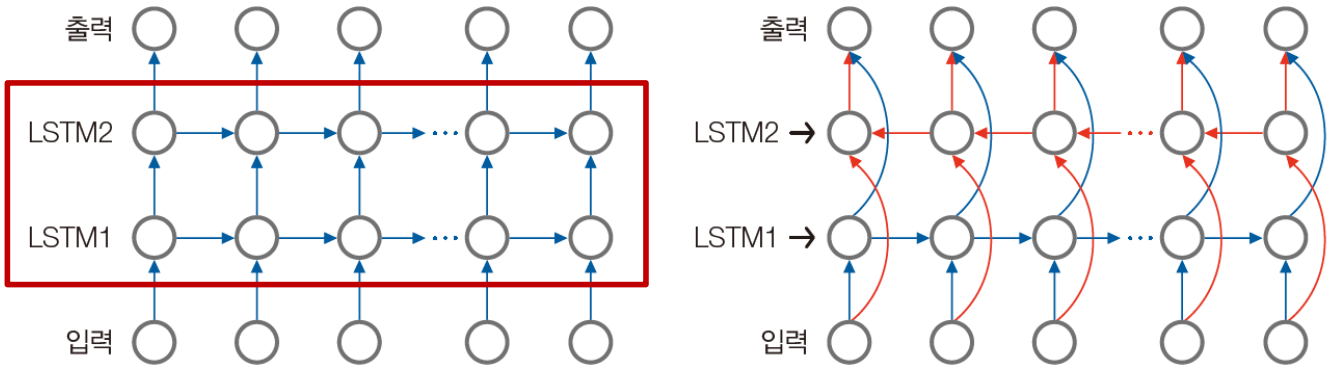
그림 8-7 순환 신경망과 LSTM

LSTM 여기 설명 부족해 보임

8.2.3 LSTM의 유연한 구조

- 장단기메모리 신경망(LSTM)은 순환 신경망(RNN) 처럼 다양하게 변형할 수 있음 ➔ 은닉층 깊게
- [그림 8-8(a)]는 은닉층(LSTM 셀층)을 2개 쌓은 적층 LSTM (stacked LSTM) 임
  - 그러나 은닉층 2개 쌓는 Deep RNN과 동일 구조 아님
- 적층 LSTM은 층을 깊게 쌓는 형태이며, 복잡도가 높아 효율적인 특징을 지님

층 2개 짜리 적층 LSTM



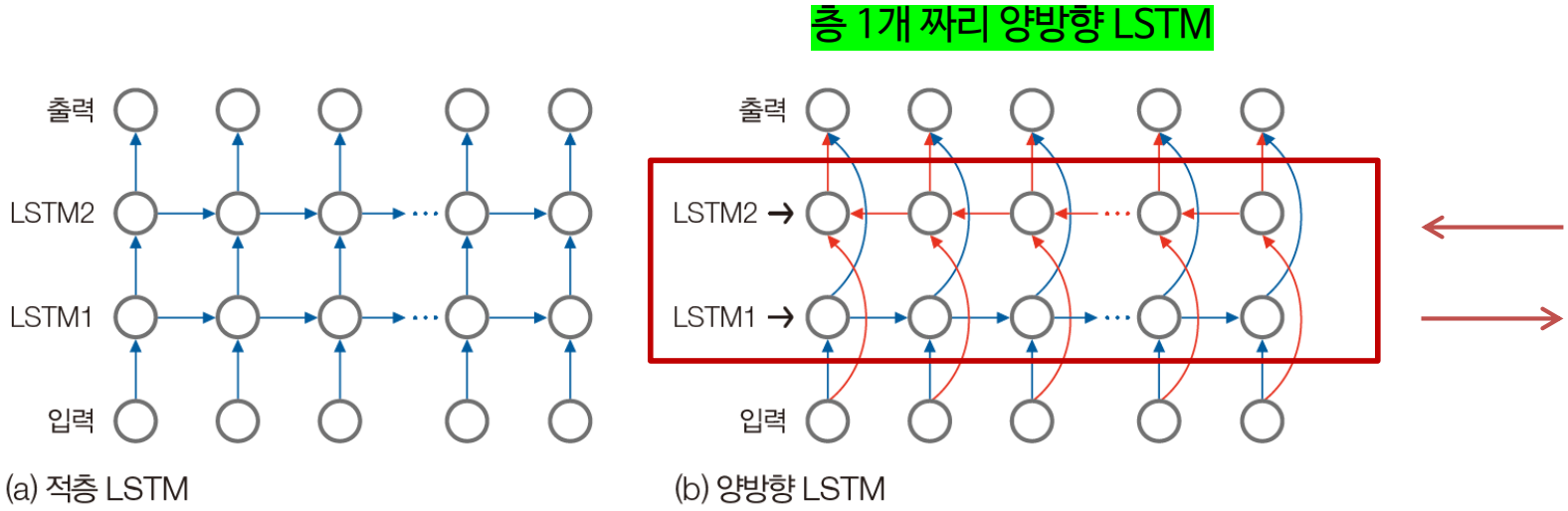
(a) 적층 LSTM (b) 양방향 LSTM

그림 8-8 적층 LSTM과 양방향 LSTM

8.2.3 LSTM의 유연한 구조

- 장단기메모리 신경망(LSTM)은 순환 신경망(RNN) 처럼 다양하게 변형할 수 있음 ➔ 양방향
- 시계열 데이터는 오른쪽 방향과 왼쪽 방향을 모두 살펴야 문맥을 제대로 파악할 수 있는 경우가 많음
  - 예) "잘 **달리는** 이 **차**는 유럽에서 생산했다" vs "고산 지대에서 생산한 이 **차**는 **향**이 좋다"

- [그림 8-8(b)]는 첫 번째 은닉층에서는 원래처럼 오른쪽으로 데이터가 흐르고 두 번째 은닉층에서는 데이터가 왼쪽으로 흐름. 이런 신경망을 양방향 LSTM (bi-directional LSTM)이라고 함

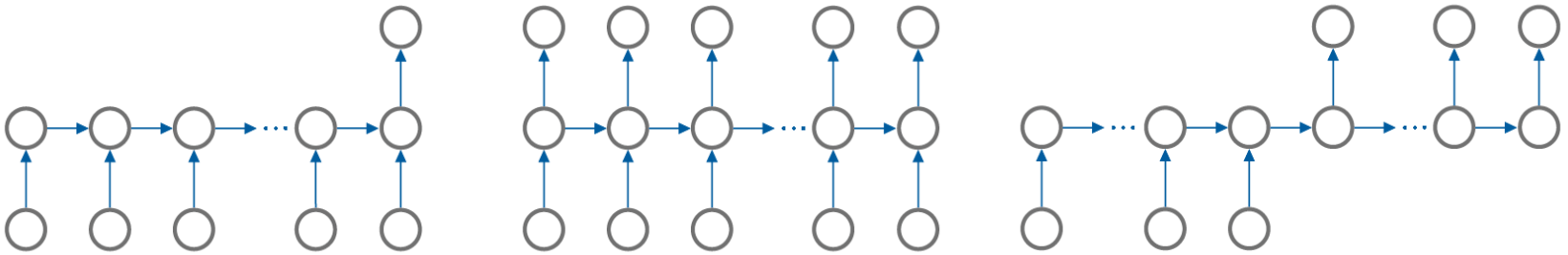


(a) 적층 LSTM (b) 양방향 LSTM

그림 8-8 적층 LSTM과 양방향 LSTM

8.2.3 LSTM의 유연한 구조

- 여러 응용 문제를 위한 다양한 LSTM 신경망 구조
  - [그림 8-9(a)] 미래 예측, 비트코인 가격 예측, 심전도 데이터를 통한 정상인 혹은 환자 예측 문제에 적합한 신경망 구조
  - [그림 8-9(b)] 비디오를 구성하는 프레임별 영상 내용 분류 응용에 적합한 신경망 구조
  - [그림 8-9(c)] 언어 번역 응용에 적합한 신경망 구조
    - 앞부분에 한국어 문장을 구성하는 단어가 순서대로 입력되면 구문과 의미를 파악하고, 뒤에서는 영어 문장을 생성해 출력하는 방식



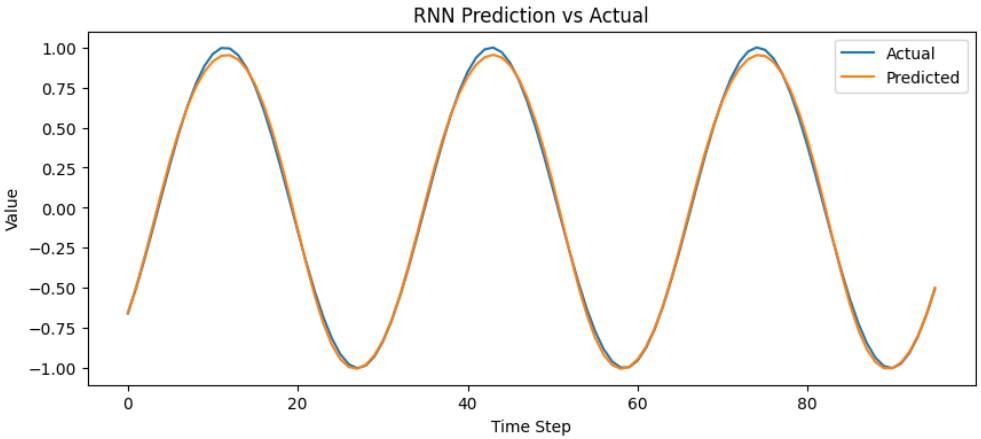
(a) 분류 또는 예측 문제 (b) 비디오 프레임 분류 (c) 언어 번역

그림 8-9 다양한 문제에 적용할 수 있는 LSTM

8.3.1 단일 채널 센서 값 예측

- [프로그램 8-2] 단일 파장 센서 예측하기

- 순환 신경망 (RNN) 을 이용한 1D 신호 예측하기



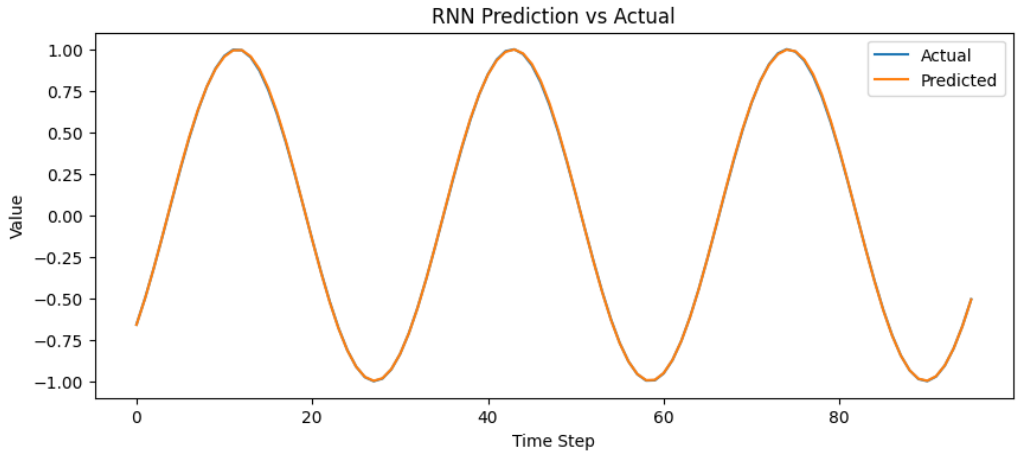


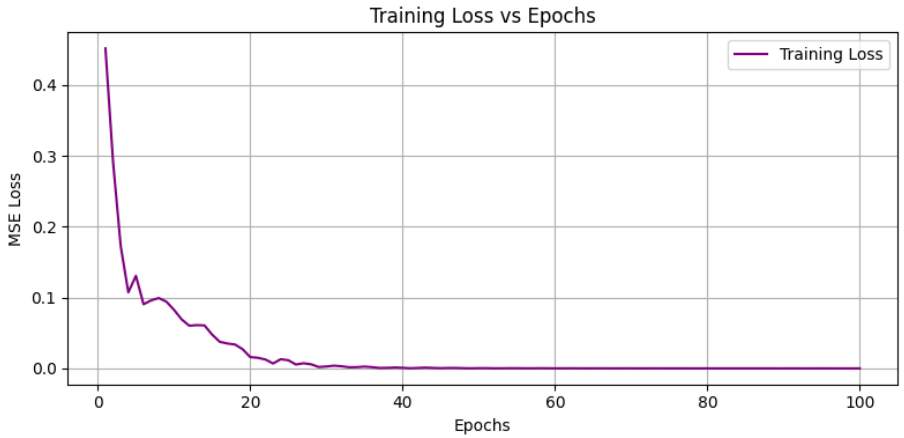
[신] https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-rnn-codingnote ← 코드 정리 제공본
[구] https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-rnn-3-codingnote

8.3.1 단일 채널 센서 값 예측

- [프로그램 8-2] 단일 파장 센서 예측하기

- 순환 신경망 (LSTM) 을 이용한 1D 신호 예측하기





https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-lstm-codingnote

8.3.1 단일 채널 비트코인 가격 예측

- [그림 8-4(b)]를 보면 비트코인 가격 데이터에는 (종가, 시가, 고가, 저가)의 네 열이 있으며, 해당 데이터는 입력으로 모두 사용될 수 있음
- **프로그램 8-2 에서는 종가만을 사용하여 입력이 1개인 단일 채널 데이터를 다루기로 함**



(a) 코인데스크에서 데이터를 다운로드하기

B	C	D	E	F	G
Date	Closing Pr	24h Open	24h High	24h Low (USD)	
2019-02-28	3772.94	3796.64	3824.17	3666.52	
2019-03-01	3799.68	3773.44	3879.23	3753.8	
2019-03-02	3811.61	3799.37	3840.04	3788.92	
2019-03-03	3804.42	3806.69	3819.19	3759.41	
2019-03-04	3782.66	3807.85	3818.7	3766.24	
2019-03-05	3689.86	3783.36	3804.35	3663.48	
2019-03-06	3832.08	3701.05	3866.72	3688.7	
2019-03-07	3848.96	3832.59	3881.97	3802.52	
2019-03-08	3859.84	3848.96	3890.75	3827.67	
2019-03-09	3828.37	3859.8	3918	3778.52	
2019-03-10	3898.87	3841.89	3948.88	3832.14	
2019-03-11	3899.66	3916.76	3921.93	3865.92	
2019-03-12	3851.25	3899.46	3913.46	3819.93	

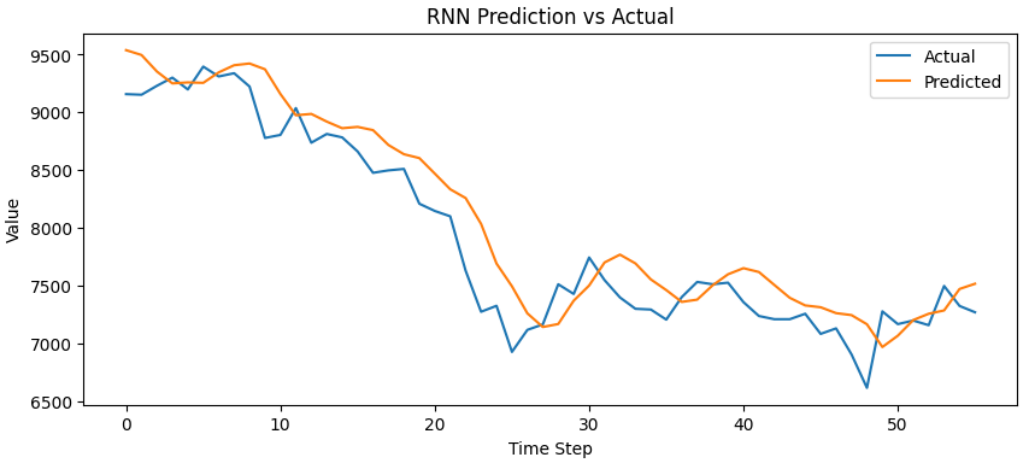
(b) 비트코인 가격이 저장된 데이터 프레임

그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측

- 순환 신경망 (RNN) 을 이용한 비트코인 종가 예측하기



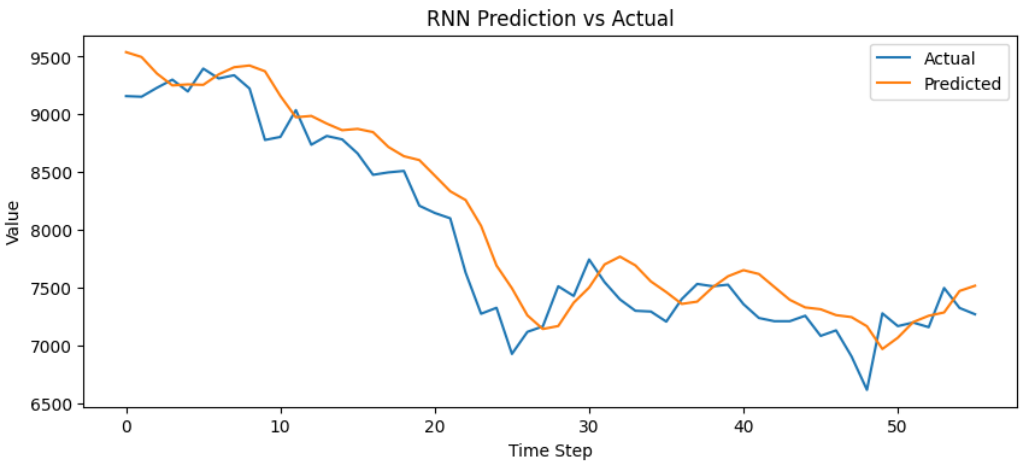


https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-rnn-2-codingnote

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측

- 순환 신경망 (LSTM) 을 이용한 비트코인 종가 예측하기





https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-lstm-2-codingnote

8.3.1 단일 채널 비트코인 가격 예측

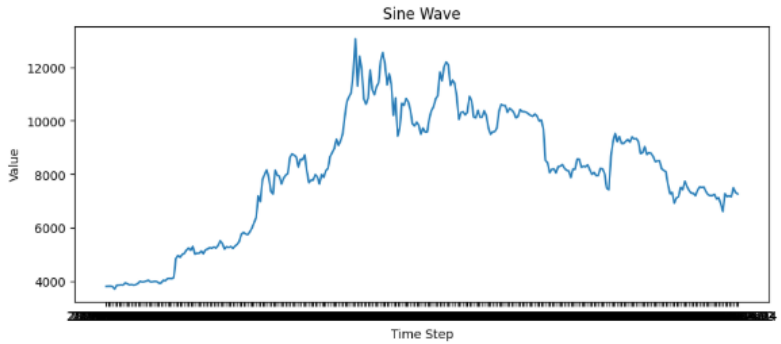
- [프로그램 8-2] 단일 채널 비트코인 가격 예측 : **(1) 파이토치 내 lstm을 사용하는 솔루션**
  - 학습 및 테스트 데이터 준비

In [5]:
```
import pandas as pd
df = pd.read_csv("/kaggle/input/bitcoin-price/bitdata.csv", index_col=0)
df['value'] = pd.DataFrame(df, columns=['closing price'])
df.head(5)
```

Out[5]:

date	closing price	open price	high price	low price	value
2019-02-28	3816.6	3814.6	3883.7	3783.3	3816.6
2019-03-01	3821.9	3816.7	3855.8	3816.4	3821.9
2019-03-02	3823.1	3821.9	3843.2	3783.6	3823.1
2019-03-03	3809.5	3823.2	3836.6	3789.7	3809.5
2019-03-04	3715.9	3809.7	3828.4	3681.8	3715.9

In [6]:
```
plt.figure(figsize=(10, 4))
plt.plot(df['value'])
plt.title("Sine Wave")
plt.xlabel("Time Step")
plt.ylabel("Value")
plt.show()
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-lstm-2-codingnote

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
  - 윈도우에 맞게 시계열 데이터 준비 (w = 20, h = 1)

데이터 정규화 반드시 필요

In [7]:
```
scaler = MinMaxScaler()

df['value'] = scaler.fit_transform(df[['value']])  # Normalize values
data = df['value'].values  # Convert to numpy array for sequence creation
```

In [8]:
```
def create_sequences(data, seq_length):
  xs, ys = [], []
  for i in range(len(data) - seq_length):
    x = data[i:i+seq_length] # Sequence of length `seq_length`
    y = data[i+seq_length] # Label is the next value
    xs.append(x)
    ys.append(y)
  return np.array(xs), np.array(ys)

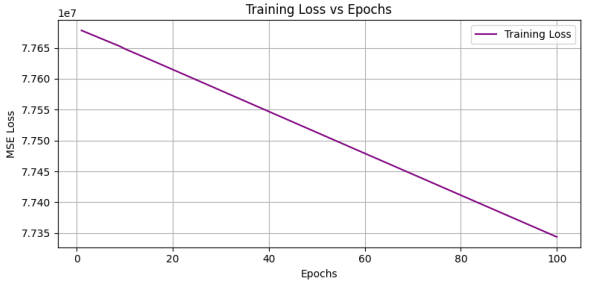
SEQ_LENGTH = 20 # Number of past time steps to look at

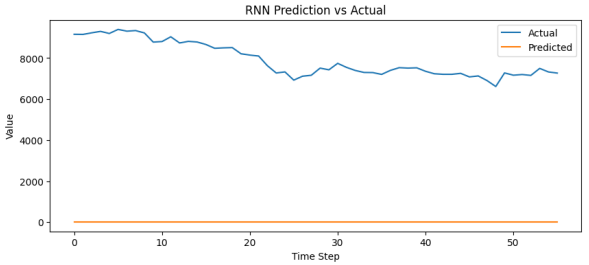
X, y = create_sequences(data, SEQ_LENGTH)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

# Convert arrays to PyTorch tensors
X_train = torch.Tensor(X_train).unsqueeze(-1) # Shape: (batch, seq, input_size)
y_train = torch.Tensor(y_train)
X_test = torch.Tensor(X_test).unsqueeze(-1)
y_test = torch.Tensor(y_test)
```

데이터 정규화 미 실행시 학습 불안정 결과





https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w11-lstm-3-codingnote

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
  - 모델 정의

In [9]:

```
import torch
import torch.nn as nn

class BasicLSTM(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=1):
        super(BasicLSTM, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)          # RNN output for all time steps
        out = out[:, -1, :]           # Take output from the last time step
        return self.fc(out)           # Pass through linear layer

model = BasicLSTM()
```

Num_layers = 2 이상이 되면, stacked LSTM 모델이 됨

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
  - 모델 학습

In [10]:

```
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

EPOCHS = 100

losses = []

for epoch in range(EPOCHS):
    model.train()
    output = model(X_train)
    loss = criterion(output.squeeze(), y_train)  # Compute loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    losses.append(loss.item())

    if (epoch + 1) % 10 == 0:
        print(f"Epoch [{epoch+1}/{EPOCHS}], Loss: {loss.item():.4f}")
```

```
Epoch [10/100], Loss: 0.0557
Epoch [20/100], Loss: 0.0263
Epoch [30/100], Loss: 0.0161
Epoch [40/100], Loss: 0.0042
Epoch [50/100], Loss: 0.0036
Epoch [60/100], Loss: 0.0034
Epoch [70/100], Loss: 0.0030
Epoch [80/100], Loss: 0.0028
Epoch [90/100], Loss: 0.0027
Epoch [100/100], Loss: 0.0026
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
  - 학습된 LSTM 모델을 이용한 테스트 데이터 예측

In [12]:

```
model.eval()
with torch.no_grad():
  predictions = model(X_test).squeeze().numpy() # Predict on test set

# Inverse transform to original scale
y_test_inv = scaler.inverse_transform(y_test.reshape(-1, 1))
predictions_inv = scaler.inverse_transform(predictions.reshape(-1, 1))


# Plot actual vs predicted
plt.figure(figsize=(10, 4))
plt.plot(y_test_inv, label='Actual')
plt.plot(predictions_inv, label='Predicted')
plt.title("RNN Prediction vs Actual")
plt.xlabel("Time Step")
plt.ylabel("Value")
plt.legend()
plt.show()
```

데이터 정규화 반드시 필요

8.3.2 성능 평가

- 평균절댓값오차mean absolute error(MAE): 스케일 문제에 대처하지 못함
  - 농산물의 판매량 예측에서 복숭아 MAE 1.8 의 결과가 수박 MAE 18 보다 성능이 뛰어나다고 말할 수 없음

$$\text{평균절댓값오차: } \mathrm{MAE} = \frac{1}{|M|}\sum_{x \in M}|y-o| \qquad (8.4)$$

- 평균절댓값백분율오차: 스케일 문제에 대처

$$\text{평균절댓값백분율오차: } \mathrm{MAPE} = \frac{1}{|M|}\sum_{x \in M}\left|\frac{y-o}{y}\right| \qquad (8.5)$$

표 8-1 평균절댓값오차와 평균절댓값백분율오차

데이터 스케일	데이터(y_test는 참값, pred는 예측값)	평균절댓값오차(MAE)	평균절댓값백분율오차(MAPE)
두 자릿수	y_test [12 20 18 22 28]	(2+1+2+2+2)/5 =1.8	(2/12+1/20+2/18+2/22+2/28)/5 =0.0980
	pred [10 21 16 20 26]		
세 자릿수	y_test [120 200 180 220 280]	(20+10+20+20+20)/5 =18	(20/120+10/200+20/180+20/220+ 20/280)/5=0.0980
	pred [100 210 160 200 260]		

8.3.2 성능 평가

- 등락 정확률
  - 등락을 얼마나 정확하게 맞히는지 측정
  - 맞힌 경우의 수를 전체 샘플 수로 나눔

표 8-2 등락 정확률

샘플 i	x_test[i]	y_test[i]	pred[i]	맞힘
1	[.., .., .., .., 21]	23 업	24 업	o
2	[.., .., .., .., 25]	20 다운	26 업	x
3	[.., .., .., .., 22]	24 업	23 업	o
4	[.., .., .., .., 28]	25 다운	26 다운	o
5	[.., .., .., .., 21]	18 다운	17 다운	o
6	[.., .., .., .., 32]	31 다운	33 업	x
7	[.., .., .., .., 35]	36 업	37 업	o
8	[.., .., .., .., 20]	19 다운	22 업	x
				등락 정확률 = 5/8 = 62.5%

8.3.3 다중 채널 비트코인 가격 예측

- [그림 8-4(b)]를 보면 비트코인 가격 데이터에는 (종가, 시가, 고가, 저가)의 네 열이 있으며, 해당 데이터는 입력으로 모두 사용될 수 있음
- 프로그램 8-3 에서는 입력이 4개인 다중 채널 데이터를 다루기로 함



(a) 코인데스크에서 데이터를 다운로드하기

	A	B	C	D	E	F	G
1	Currency	Date	Closing Pr	24h Open	24h High	24h Low (USD)	
2	BTC	2019-02-28	3772.94	3796.64	3824.17	3666.52	
3	BTC	2019-03-01	3799.68	3773.44	3879.23	3753.8	
4	BTC	2019-03-02	3811.61	3799.37	3840.04	3788.92	
5	BTC	2019-03-03	3804.42	3806.69	3819.19	3759.41	
6	BTC	2019-03-04	3782.66	3807.85	3818.7	3766.24	
7	BTC	2019-03-05	3689.86	3783.36	3804.35	3663.48	
8	BTC	2019-03-06	3832.08	3701.05	3866.72	3688.7	
9	BTC	2019-03-07	3848.96	3832.59	3881.97	3802.52	
10	BTC	2019-03-08	3859.84	3848.96	3890.75	3827.67	
11	BTC	2019-03-09	3828.37	3859.8	3918	3778.52	
12	BTC	2019-03-10	3898.87	3841.89	3948.88	3832.14	
13	BTC	2019-03-11	3899.66	3916.76	3921.93	3865.92	
14	BTC	2019-03-12	3851.25	3899.46	3913.46	3819.93	

(b) 비트코인 가격이 저장된 데이터 프레임

그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

다중 채널 비트코인 가격 예측 실습 과제

- 제출 방식 : 이메일로 보고서 제출
  - 마감 : 11월 26일 (수) 23시 59분
  - 이메일 주소 : admin@rcv.sejong.ac.kr
  - 이메일 제목 : [2025-2] [인공지능] 11주차 과제 제출 (학번_이름)
  - 보고서 분량 제한 없음

- [문제1] 다중 채널 비트코인 가격 예측
  - 단일 채널 비트코인 가격 예측 문제 실습을 기반으로 해결 하시오
  - RNN, LSTM 기본 모델의 성능을 비교하시오
  - https://www.kaggle.com/t/14c91f8ce42641df88f19ea4bf5e24fe

8.5 자연어 처리

- 자연어 처리(NLP: natural language processing)란?
  - 인간이 구사하는 언어를 자동으로 처리하는 프로그램을 만드는 연구 분야

- 자연어 처리 응용 분야
  - 한국어를 영어로 번역하는 기술
  - 영화평에 달린 댓글을 분석해 흥행을 추정하는 기술
  - 고객의 음성에서 의도를 이해하여 적절히 응대하는 챗봇 기술
  - 소설이나 시를 쓰는 창작 인공지능 기술

8.5.1 텍스트 데이터에 대한 이해

- **영화평 데이터셋 IMDB의 예시**

> Once again Mr. Costner has dragged out a movie for far longer than necessary. Aside from the terrific sea rescue sequences, of which there are very few I just did not care about any of the characters. Most of us have ghosts in the closet, and Costner's character are realized early on, and then forgotten until much later, by which time I did not care. ……

- 텍스트 데이터의 특성
  - 시계열 데이터로서 시간 정보가 있고 샘플마다 길이가 다름
  - 심한 잡음
    - 숫자, 특수 기호, 외국어, 비속어(ㅋㅋㅋ), 비문, 사투리, 오타, 띄어쓰기 오류 등
  - 형태소 분석 필요
    - 형태소란 더 나누면 의미가 훼손되는 문장의 최소 단위
    - 한글은 다른 언어에 비해 형태소 분석이 어려움
    - 예시) "너는 누구에게 한 번이라도" ➔ "너+는 누구+에게 한 번+이라도"

8.5.1 텍스트 데이터에 대한 이해

- 텍스트 데이터의 특성
  - 다양한 언어 특성
    - 성별유무 : 영어에는 명사에 성별이 없지만, 독일어에는 명사에 성별이 있음
  - 신경망에 입력하려면 기호를 수치로 변환 해야함
    - 신경망은 수치 값을 입력으로 받기 때문에 원핫 코드나 적당한 코드로 변환 필요
    - 현대 딥러닝은 단어 임베딩 기술을 사용해 벡터 공간으로 변환
  - 구문론과 의미론
    - 구문론 : 형태소 분석처럼 문법적인 구조를 따지는 일
    - 의미론: 각 단어의 의미, 단어와 단어가 결합해 만들어 내는 의미를 이해하는 일
    - 의미론까지 분석해야 똑똑한 챗봇을 만들 수 있음

8.5.1 텍스트 데이터에 대한 이해

- 원핫 코드 표현
  - 말뭉치(Corpus) 사례
    - 특정한 목적을 가지고 수집한 문장의 집합을 말뭉치corpus 라고 함

말뭉치 예)

```
Freshman loves python.
We teach python to freshman.
How popular is Python?
```

  - 단어 빈도를 통한 원핫 코드 표현 (괄호 숫자, 빈도수)
    - python(3), freshman(2), loves(1), we(1), teach(1), to(1), how(1), popular(1), is(1)

8.5.1 텍스트 데이터에 대한 이해

- 원핫 코드 표현
  - 말뭉치(Corpus) 사례
    - 특정한 목적을 가지고 수집한 문장의 집합을 말뭉치corpus 라고 함

말뭉치 예)

```
Freshman loves python.
We teach python to freshman.
How popular is Python?
```

  - 파이썬의 자료구조인 딕셔너리를 이용한 표현(빈도수에 따른 순위 부여)
    - {'python':1, 'freshman':2, 'loves':3, 'we': 4, 'teach': 5, 'to': 6, 'how': 7, 'popular':8, 'is': 9}
  - 딕셔너리를 이용하여 텍스트를 숫자 코드로 변환
    - Freshman loves python. ➔ [2 3 1]
    - We teach python to freshman. ➔ [4 5 1 6 2]
    - How popular is python? ➔ [7 8 9 1]

가변길이는 고정길이로 맞추기 위해 padding 처리 필요

8.5.1 텍스트 데이터에 대한 이해

- 원핫 코드 표현
  - 딕셔너리 수치 표현 [2 3 1] 이나 [4 5 1 6 2]를 그대로 신경망에 입력하지 않고 **원핫 코드** 또는 **보다 발전된 표현 방법인 단어 임베딩**으로 변환한 후 신경망 입력으로 사용함
  - 원핫 코드 예

문장 내 〈사전 단어〉가 있다/없다로 표현

```
[231]   →  [[010000000] [001000000] [100000000]]
[45162] →  [[000100000] [000010000] [100000000] [000001000] [010000000]]
[7891]  →  [[000000100] [000000010] [000000001] [100000000]]
```

- 원핫 코드의 문제점
  - 사전 크기가 크면 원핫 코드는 **희소 벡터가 되어 메모리 낭비가 심해짐**
  - **단어 사이의 연결 관계를 반영하지 못함**
    - 예) king과 queen, bridegroom과 bride의 관계 표현 불가능
  - 현대적인 표현 기법 단어 임베딩은 나중에 다룸 (Word2Vec, Glove 등)
    - 단어의 의미 관계를 훌륭하게 반영

8.5.1 텍스트 데이터에 대한 이해

- 통계적 수치 기반 표현
  - TF (Term Frequency)
    - 단어가 문서 안에서 몇 번 등장 했는지를 나타내는 통계적 빈도값 표현 방식
  - TF-IDF (Term Frequency X Inverse Document Frequency)
    - 단어 빈도와 역문서 빈도를 조합하여, 문서 내에서 중요한 단어에 높은 가중치를 부여하는 표현 방식

- 학습기반 임베딩 표현
    - 단어 의미와 문맥을 살린 표현 방식
    - 학습을 통해 의미 공간을 학습하는 방식
    - 예) Word2Vec, Glove 등

8.5.1 텍스트 데이터에 대한 이해

- 학습기반 임베딩 표현
  - Word2Vec
    - 문맥 기반 임베딩 기술
      - 비슷한 컨텍스트를 갖는 단어의 경우 임베딩도 유사해야 한다는 컨셉
      - 주변 단어를 예측하는 방식(CBoW, Skip-gram)으로 학습 진행
      - 대규모 데이터에서 잘 동작

예를 들어 “good”이라는 단어의 주변에는 “great, excellent, amazing…” 같은 긍정적 단어들이 자주 등장하므로, Word2Vec은 이 단어들을 임베딩 공간에서 가깝게 위치시키게 됨.

반대로, “bad” 주변에는 “terrible, awful, disappointing…” 같은 부정적 표현들이 자주 등장하며, 해당 단어들도 임베딩 공간에서 가깝게 위치시키게 됨.

  - Glove
    - 관계 기반 임베딩
      - 코퍼스 전체에서 단어 간 동시 등장 횟수를 계산
        - 예) hot-gas는 steam에 많이 등장, cold-solid는 ice에 많이 등장
      - 수학적 해석성이 높고 안정적인 벡터 생성

8.5.1 텍스트 데이터에 대한 이해

- 텍스트를 수치화 하는 방법 (text vectorization)

구분	원핫 표현	통계 기반 표현	학습기반 임베딩 표현
벡터 형태	Sparse (고차원)	Sparse (고차원)	Dense(저차원)
값의 의미	단어 존재 여부	빈도/중요도	의미/맥락
학습 방식	규칙 기반	규칙 기반	학습 기반
차원 수	*단어 사전 크기	*단어 사전 크기	일반적으로 50~1024
대표적 알고리즘	One-hot	TF, TF-IDF, BoW	Word2Vec, GloVe, BERT
문맥 고려 여부	X	X	일부 고려
메모리 효율	매우 비효율적	비효율적	효율적
의미적 유사성 반영	X	X	O
실제 사용 트렌드	거의 사용 안 함	초기에만 사용	최신 NLP 핵심

*단어 사전 (vocabulary) : NLP에서 분석에 사용되는 유효 단어들의 집합. 단어 사전의 크기에 따라 유효 단어의 수를 제한 한 것.

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 영화를 평가한 댓글을 모아둔 IMDB 데이터셋
  - 50000개의 댓들을 긍정 평가와 부정 평가로 레이블링
  - 감정 분류(sentiment classification) 문제에 주로 사용

- IMDB 데이터 살펴보기
  - https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p1-codingnote

[3]:
```
nRowsRead = 1000
df = pd.read_csv('/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv', delimiter=',', nrows = nRowsRead)
```

[4]:
```
df.head(5)
```

[4]:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 데이터 살펴보기
  - https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p1-codingnote

```
df['review'][0]
```

```
"One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me.<br /><br />The fir
st thing that struck me about Oz was its brutality and unflinching scenes of violence, which set in right from the word GO. Trust me, this is not a show for the faint hearted
or timid. This show pulls no punches with regards to drugs, sex or violence. Its is hardcore, in the classic use of the word.<br /><br />It is called OZ as that is the nickna
me given to the Oswald Maximum Security State Penitentary. It focuses mainly on Emerald City, an experimental section of the prison where all the cells have glass fronts and
face inwards, so privacy is not high on the agenda. Em City is home to many..Aryans, Muslims, gangstas, Latinos, Christians, Italians, Irish and more....so scuffles, death st
ares, dodgy dealings and shady agreements are never far away.<br /><br />I would say the main appeal of the show is due to the fact that it goes where other shows wouldn't da
re. Forget pretty pictures painted for mainstream audiences, forget charm, forget romance...OZ doesn't mess around. The first episode I ever saw struck me as so nasty it was
surreal, I couldn't say I was ready for it, but as I watched more, I developed a taste for Oz, and got accustomed to the high levels of graphic violence. Not just violence, b
ut injustice (crooked guards who'll be sold out for a nickel, inmates who'll kill on order and get away with it, well mannered, middle class inmates being turned into prison
bitches due to their lack of street skills or prison experience) Watching Oz, you may become comfortable with what is uncomfortable viewing....thats if you can get in touch w
ith your darker side."
```

+ Code + Markdown

```
df['sentiment'][0]
```

```
'positive'
```

```
df['review'][3]
```

```
"Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his parents are fighting all the time.<br /><br />This movie is slower than a so
ap opera... and suddenly, Jake decides to become Rambo and kill the zombie.<br /><br />OK, first of all when you're going to make a film you must Decide if its a thriller or
a drama! As a drama the movie is watchable. Parents are divorcing & arguing like in real life. And then we have Jake with his closet which totally ruins all the film! I expec
ted to see a BOOGEYMAN similar movie, and instead i watched a drama with some meaningless thriller spots.<br /><br />3 out of 10 just for the well playing parents & descent d
ialogs. As for the shots with Jake: just ignore them."
```

```
df['sentiment'][3]
```

```
'negative'
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **텍스트 데이터 수치 표현 방법론**
  - sklearn 라이브러리 TF(Term Frequency) 방법론
    - 단어의 등장 횟수에 비례하여 가중치 부여하는 방법론

```
from sklearn.feature_extraction.text import CountVectorizer

one_hot_vectorizer = CountVectorizer()
one_hot = one_hot_vectorizer.fit_transform(x_train).toarray() # 원-핫 백터
one_hot_vectorizer.get_feature_names_out() # 학습된 단어 모음
one_hot_vectorizer.vocabulary_ # 단어당 출현 빈도
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **텍스트 데이터 수치 표현 방법론**
  - sklearn 라이브러리 TF-IDF 방법론
    - 빈도가 높지만 문서의 특징 정보가 잘 담기지 않는 단어
    - 빈도가 낮지만 문서의 특징 정보를 잘 담고 있는 단어
    - 이런 상황을 고려한 역문서 빈도

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(x_train).toarray() # TF-IDF 벡터
vocab = tfidf_vectorizer.get_feature_names_out() # 학습된 단어 모음
tfidf_vectorizer.vocabulary_ # 단어당 출현 빈도
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **TF 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - (1) 데이터 로드 – (2)데이터 가공

[12]:
```
# imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv
df = pd.read_csv('/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv', nrows = 10000)
df.head(5)
```

[12…

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

[13]:
```
df['review'] = df['review'].astype(str)
X = df['review']
```

[14]:
```
Y = df['sentiment']
Y[df['sentiment']=='positive'] = True
Y[df['sentiment']=='negative'] = False
Y = Y.astype(str)
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p2-codingnote

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **TF 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - **(3) 특징 추출**

[5]:
```
# 텍스트 데이터 BOW로 변환
from sklearn.feature_extraction.text import CountVectorizer

BOW = CountVectorizer(binary=False,max_features=20000)
X = BOW.fit_transform(X).toarray()
X = X.astype(np.int8)
X.shape
```

입력데이터를 pytorch 모델에 넣을 거라면,
toarray()를 통해서 sparse matrix를 dense matrix로 변경해야함

[5]: (10000, 20000)

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **TF 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - **(4) 모델 학습 및 평가**

[6]:
```
from sklearn.model_selection import train_test_split

train_X, test_X, train_Y, test_Y = train_test_split(X, Y, test_size = 0.3, random_state = 42)
```

[7]:
```
from sklearn.linear_model import  SGDClassifier

model = SGDClassifier(random_state=42)
model.fit(train_X,train_Y)
```

[7]:
```
SGDClassifier
SGDClassifier(random_state=42)
```

[8]:
```
print(f'Train score:{format(model.score(train_X,train_Y))}',end ='\n\n')
print(f'Test score:{format(model.score(test_X,test_Y))}',end = '\n\n')
```

```
Train score:0.8225714285714286

Test score:0.7653333333333333
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **TF-IDF 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - **(3) 특징 추출**

[5]:
```
# 텍스트 데이터 TF-IDF로 변환
from sklearn.feature_extraction.text import TfidfVectorizer


tfidf = TfidfVectorizer(max_features=20000)
X = tfidf.fit_transform(X).toarray()
X.shape
```

←TF방법론의 차이

[5]:
```
(10000, 20000)
```

  - **(4) 모델 학습 및 평가**

[9]:
```
print(f'Train score:{format(model.score(train_X,train_Y))}',end ='\n\n')
print(f'Test score:{format(model.score(test_X,test_Y))}',end = '\n\n')
```

```
Train score:0.9825714285714285

Test score:0.872
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p3-codingnote

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **텍스트 데이터 수치 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - TF-IDF 방법론 + MLP 분류기 (분류기 변경)
  - 샘플사이즈 =100, MLP hidden = 256

[20]:

```
# ======================
# 4. MLP 모델 정의 (심플 버전)
# ======================
class MLPClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim=256, num_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(hidden_dim, num_classes)
        )

    def forward(self, x):
        return self.net(x)

input_dim = X_train_tensor.shape[1]
model = MLPClassifier(input_dim=input_dim, hidden_dim=256, num_classes=2)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# ======================
# 5. 손실함수 & 옵티마이저
# ======================
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
Test Accuracy: 74.33%
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p4-codingnote

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- **텍스트 데이터 수치 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - TF-IDF 방법론 + LSTM 분류기 (분류기 변경)
    - LSTM방법론은 데이터 가공 필요
  - 샘플의 크기 = 100, LSTM 입력 채널 = 100

[51]:

```python
# ============================
# 4. LSTM 분류 모델 정의
# ============================
class LSTMClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim=128, num_layers=1, num_classes=2):
        super().__init__()
        # 여기서는 embedding 계층이 필요 없음 (이미 TF-IDF 실수 벡터)
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        # x: (batch, seq_len=1, input_dim)
        # out: (batch, seq_len, hidden_dim)
        # h_n: (num_layers, batch, hidden_dim)
        out, (h_n, c_n) = self.lstm(x)
        last_hidden = h_n[-1]          # (batch, hidden_dim)
        logits = self.fc(last_hidden)  # (batch, num_classes)
        return logits
```

시계열 데이터의 특성을 반영한 LSTM이 우월한 성능을 보이지 못했다.

왜 일까? 부정 평가와 긍정 평가로 구분하는 문제에서는 특정한 단어가 얼마나 자주 등장하는지가 더 중요한 것으로 보인다.

```
Test Accuracy: 73.86%
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p6-codingnote

8.5.3 단어 임베딩

- 텍스트 데이터를 신경망에 입력하고 표현하는 방법
  - 원핫 코드 (One-hot Code)
    - 사전이 매우 커서 아주 큰 희소 텐서가 입력됨
    - 단어의 의미를 전혀 반영하지 못함

[원핫 코드]

사전 크기([프로그램 8-7(a)]의 dic_siz)가 10000인 경우

```
                          10000개 요소
"python" → (1 0 0 0 0 … … 0 0 0 … 0 0 0)
```

  - 단어 임베딩 (Word Embedding)
    - 저차원 공간의 밀집 벡터로 표현하는 기법
    - 학습을 통해 단어의 의미를 반영 가능함

[단어 임베딩]

단어 임베딩 공간의 차원([프로그램 8-7(b)]의 embed_space_dim)이 16인 경우

```
                    16개 요소
"python" →  (0.01 0.23 0.0 … 0.002)
```

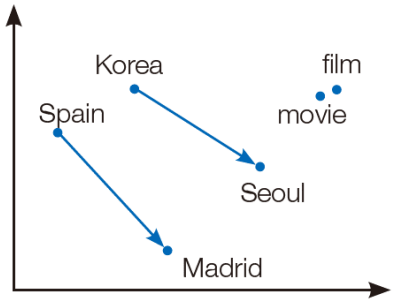


그림 8-17 단어 임베딩의 원리

8.5.3 단어 임베딩

- 텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기
  - 단어 단위 임베딩 Word2Vec
    - Gensim 라이브러리 사용하기 (사전학습 임베딩 모델 불러오기)

```
# https://wikidocs.net/233744
# 자연어 처리 라이브러리 : gensim - 단어 단위임베딩 모델

import gensim
from gensim.models import Word2Vec
from gensim.models import KeyedVectors

# binary=True 옵션 필수 (Google News 모델은 binary format)
model_path = "/kaggle/input/word2vec/pytorch/default/1/GoogleNews-vectors-negative300.bin"
word2vec = KeyedVectors.load_word2vec_format(model_path, binary=True)
```

+ Code + Markdown

```
[2]:
# 단어 vector 얻기 - 300차원의 벡터
vector = word2vec["king"]

# 단어 간 유사도
similarity = word2vec.similarity("king", "queen")

# 가장 유사한 단어
result = word2vec.most_similar("computer", topn=5)
```

```
[3]: vector.shape
[3]: (300,)

[4]: similarity
[4]: 0.6510956

print(result)
[('computers', 0.7979379892349243), ('laptop', 0.6640493273735046), ('laptop_computer', 0.6548868417739868), ('Computer', 0.647333562374115), ('com_puter', 0.6082080006599426)]
+ Code + Markdown
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-codingnote

8.5.3 단어 임베딩

- 텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기
  - 단어 단위 임베딩 Word2Vec 을 imdb로 학습하여 사용하기
    - 문장 임베딩은 어떻게? 문장 내 단어 벡터의 평균이라는 가장 쉬운 방법!

```
# tokens = ["this", "movie", "is", "great"]
# w2v["this"] → 100차원 벡터
# w2v["movie"] → 100차원 벡터
# w2v["is"] → 100차원 벡터
# w2v["great"] → 100차원 벡터

sentence_vector = mean([vec("this"), vec("movie"), vec("is"), vec("great")])
```

[8]:

```
########################################
# 5. 문장을 평균 Word2Vec 벡터로 변환
########################################
def sentence_to_vec(tokens, w2v, dim):
    vecs = []
    for w in tokens:
        if w in w2v:
            vecs.append(w2v[w])
    if len(vecs) == 0:
        return np.zeros(dim, dtype=np.float32)

    return np.mean(np.stack(vecs, axis=0), axis=0).astype(np.float32)
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-2-codingnote

8.5.3 단어 임베딩

- 텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기
  - 단어 단위 임베딩 Word2Vec 을 imdb로 학습하여 사용하기
    - tokenizer 는 문장을 개별 단어(토큰)으로 나누는 역할을 담당
      - 즉, 텍스트를 단어 리스트로 변환하는 과정

```
########################################
# 6. Dataset 정의
########################################
class IMDBW2VDataset(Dataset):
    def __init__(self, dataframe, w2v_model):
        self.df = dataframe
        self.w2v = w2v_model
        self.dim = w2v_model.vector_size

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        text = self.df.loc[idx, "review"]
        sentiment = self.df.loc[idx, "sentiment"]

        tokens = tokenizer(text)
        vec = sentence_to_vec(tokens, self.w2v, self.dim) # 평균 벡터임

        x = torch.tensor(vec, dtype=torch.float32)
        y = torch.tensor(1.0 if sentiment == "positive" else 0.0, dtype=torch.float32)

        return x, y

# 즉, train_dataset을 단어 단위 임베딩으로 어떻게 만들었는지 봐야함
train_dataset = IMDBW2VDataset(train_df, w2v_model)
test_dataset = IMDBW2VDataset(test_df, w2v_model)
```

입력	tokenizer	출력
`"This movie is amazing!"`	`tokenizer(text)`	`["this", "movie", "is", "amazing!"]`

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-2-codingnote

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - **단어** 단위 임베딩 Word2Vec 을 imdb로 학습하여 사용하기
  - Word2Vec+MLP 분류기 학습 결과 : 정확도 84%

```
[Epoch 1] Train Loss: 0.6022, Train Acc: 0.6936 | Test Loss: 0.4875, Test Acc: 0.8054
[Epoch 2] Train Loss: 0.4386, Train Acc: 0.8124 | Test Loss: 0.4052, Test Acc: 0.8243
[Epoch 3] Train Loss: 0.3965, Train Acc: 0.8292 | Test Loss: 0.3797, Test Acc: 0.8361
[Epoch 4] Train Loss: 0.3829, Train Acc: 0.8347 | Test Loss: 0.3731, Test Acc: 0.8383
[Epoch 5] Train Loss: 0.3772, Train Acc: 0.8373 | Test Loss: 0.3661, Test Acc: 0.8405
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-2-codingnote

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - **단어 단위 임베딩 Word2Vec+LSTM 을 imdb로 학습하여 사용하기**
    - Word2Vec+MLP와 차이는 입력 데이터 처리!

- 이전:
  - `sentence_to_vec` 로 `(dim,)` 평균 임베딩 → **MLP** 입력
- 지금:
  - `sentence_to_seq` 로 `(seq_len, dim)` 시퀀스 임베딩 → **LSTM** 입력
  - `pad_sequence` 로 배치 내 길이 맞춤
  - 마지막 hidden state를 받아서 `fc` 로 이진 분류

👉 Word2Vec 학습된 벡터를 사용한다고 가정 (차원 `100`)

문장	단어 벡터 시퀀스	실제 shape
`"I love dogs"`	`[v5, v10, v20]`	`(3, 100)`
`"Dogs are great"`	`[v20, v3, v8]`	`(3, 100)`
`"Yes"`	`[v15]`	`(1, 100)`

🛠️ 패딩 후:

문장	패딩된 벡터 시퀀스	최종 shape
`[v5, v10, v20]`	`[v5, v10, v20]`	`(3, 100)`
`[v20, v3, v8]`	`[v20, v3, v8]`	`(3, 100)`
`[v15]`	`[v15, 0,…,0], [0], [0]`	`(3, 100)`

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-5-codingnote

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - BERT 사전학습 언어모델 불러와 Fine-tuning 하기

구분	Word2Vec (단어 임베딩)	BERT (문맥적 임베딩)
임베딩 단위	단어(word) 단위	문장(sentence or subword) 단위
벡터 값	항상 동일	문맥에 따라 달라짐 ex) "not good" vs "good job"
문맥 이해	없음	있음
순서 정보	고려 안 함	고려됨
입력 표현 방식	토큰 → 평균 또는 합	Transformer self-attention으로 관계 계산
대표 모델	Word2Vec, GloVe, FastText	BERT, RoBERTa, ELECTRA, GPT
Pretraining 규모	~MB - GB	수십~수백 GB 수준 (대용량)
부정 표현 처리	“not good” → “not” + “good” 평균 → 긍정으로 해석될 수도	“good” 단어의 의미 반전됨
다운스트림 학습	보통 별도 모델 필요 (MLP/LSTM 등)	fine-tuning 가능 <= 사전학습의 의미 시작
GPU 필요	CPU도 충분	GPU 필수
권장 상황	빠른 baseline 실험, 자원 제한, 간단 NLP	고성능 요구, 의미 이해, 감정 분석, QA 등

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-4-codingnote

8.5.3 단어 임베딩

- 텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기
  - BERT 사전학습모델 불러와 Fine-tuning 하기

[3]:
```
########################################
# 3. BERT 토크나이저 & 분류 모델 로드
########################################
MODEL_NAME = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# num_labels=2 → positive / negative
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2
).to(device)
```

```
########################################
# 4. Dataset 정의 (BERT 입력용)
########################################
class IMDBBERTDataset(Dataset):
    def __init__(self, dataframe, tokenizer, max_len=256):
        self.df = dataframe
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        text = str(self.df.loc[idx, "review"])
        sentiment = self.df.loc[idx, "sentiment"]
        label = 1 if sentiment == "positive" else 0

        enc = self.tokenizer(
            text,
            padding='max_length',
            truncation=True,
            max_length=self.max_len,
            return_tensors='pt'
        )
        # enc["input_ids"]: (1, max_len)
        input_ids = enc["input_ids"].squeeze(0)       # (max_len,)
        attention_mask = enc["attention_mask"].squeeze(0)  # (max_len,)

        return {
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "label": torch.tensor(label, dtype=torch.long)
        }

max_len = 256
train_dataset = IMDBBERTDataset(train_df, tokenizer, max_len=max_len)
test_dataset  = IMDBBERTDataset(test_df, tokenizer, max_len=max_len)
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-4-codingnote

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - BERT 사전학습모델 불러와 Fine-tuning 하기
    - 5 에포크 후 성능 92.17%

[8]:

```
##########################################
# 7. 학습 루프 & 테스트 루프
# - 지금 모델의 분류기 헤드가 학습되어 있지 않은 상태라서, fine-tuning 해야 함
##########################################
num_epochs = 5

for epoch in range(1, num_epochs + 1):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer)
    test_loss, test_acc   = eval_epoch(model, test_loader, criterion)

    print(f"[Epoch {epoch}] "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.4f}")
```

```
[Epoch 1] Train Loss: 0.2598, Train Acc: 0.8924 | Test Loss: 0.2008, Test Acc: 0.9202
[Epoch 2] Train Loss: 0.1373, Train Acc: 0.9500 | Test Loss: 0.2309, Test Acc: 0.9190
[Epoch 3] Train Loss: 0.0690, Train Acc: 0.9767 | Test Loss: 0.2568, Test Acc: 0.9186
[Epoch 4] Train Loss: 0.0388, Train Acc: 0.9876 | Test Loss: 0.3289, Test Acc: 0.9114
[Epoch 5] Train Loss: 0.0308, Train Acc: 0.9899 | Test Loss: 0.3292, Test Acc: 0.9217
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w13-p10-4-codingnote

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - 다양한 조합 실험 가능

방식	성능	장점
Word2Vec 평균	★★	단순, 빠름
TF-IDF + Word2Vec 평균	★★★	중요도 반영
Word2Vec + LSTM	★★★★	순서 반영
BERT + FineTurning	★★★★★	End to End 모델

8.5.3 단어 임베딩

- **텍스트 데이터 임베딩 표현 방법론를 이용한 영화 리뷰 감정 분석 하기**
  - 텍스트 분야의 End-to-End 모델

모델	NLP E2E 여부	특징
Word2Vec (Skip-gram)	X	단어 벡터만 학습
BERT (Encoder only)	제한적	E2E보단 표현 학습 모델
Seq2Seq (2014)	E2E 원조	번역, 문장 생성
Seq2Seq + Attention (2015)	향상된 E2E	긴 문장도 처리
Transformer (2017) (Encoder-Decoder)	최종판	현재 대부분 번역 방식
GPT (Decoder only)	완전 E2E	"문맥 → 텍스트 생성"

영화평 데이터셋 IMDB 긍/부정 리뷰 예측 실습 과제

- 제출 방식 : 이메일로 보고서 제출
  - 마감 : 12월 01일 (월) 23시 59분
  - 이메일 주소 : admin@rcv.sejong.ac.kr
  - 이메일 제목 : [2025-2] [인공지능] 13주차 과제 제출 (학번_이름)
  - 보고서 분량 제한 없음

- [문제1] 긍정 부정 리뷰 예측
  - 배운 내용을 다양하게 적용하여 문제를 해결하시오.
  - https://www.kaggle.com/t/36d87d8543924314af7568964ffacb03
  - [주의] 답안 제출 시 positive, negative 로 제출해야함

	A	B
1	id	sentiment
2	0	negative
3	1	negative
4	2	negative
5	3	positive
6	4	negative
7	5	negative
8	6	negative
9	7	negative
10	8	negative
11	9	negative
12	10	negative
13	11	negative

8.5.5 word2vec 과 Glove

- **단어를 벡터 공간에 표현하는 단어 임베딩 기술**
  - 오래 전부터 연구되어온 아이디어
  - 고전적인 기법들
    - 1960년대 : TF (term frequency) 단어가 출현하는 빈도를 세어 벡터로 표현
    - 1980년대 : LSA (latent sematic analysis) 행은 단어이고 열은 문서 번호인 행렬에 단어 빈도를 채운 다음 SVD 기법을 이용해 정보 손실을 최소로 유지한 채 차원 축소를 달성하는 기법
    - 2000년대 : 신경망 기법들
    - 2010년대 : 딥러닝 기법들
      - 구글에서 공개한 word2vec
        - 1000억 개 가량의 **뉴스를 모아둔 데이터셋**으로 학습, 300만 개 가량의 단어를 각각 300차원 공간에 표현
      - 스탠퍼드에서 공개한 Glove
        - **위키피디아 문서 데이터**를 사용해 학습, 40만 개 가량의 단어를 50차원, 100차원, 200차원, 300차원 공간에 표현

8.5.5 word2vec 과 Glove

- **단어를 벡터 공간에 표현하는 단어 임베딩 기술**
  - LSA와 같은 카운트 기반 임베딩 기법은 코퍼스의 전체적인 통계 정보를 고려하기 때문에 [왕:남자=여왕:? (정답은 여자)] 와 같은 단어 의미의 유추 작업에는 성능이 떨어짐
  - Word2Vec 와 같은 예측 기반 임베딩 기법은 단어 간 유추 작업은 뛰어나지만, 임베딩 벡터가 윈도우 크기 내에서만 주변 단어를 고려하기 때문에 코퍼스의 전체적인 통계 정보를 반영하지 못함
  - Glove와 같은 학습 기반 임베딩 기법은 앞서 소개한 두 임베딩 기법의 한계를 극복하기 위해 제안되었음

- **학습 기반 임베딩 기법**
  - 사전 훈련된 단어 임베딩 (Pretrained word embedding)
  - 구글 뉴스, 위키피디아, 커먼 크롤 과 같은 대규모 말뭉치로 학습된 모델
  - 대표적인 방법론은 스탠퍼드에서 공개한 Glove (Global Vectors for Word Representation)
    - 핵심 아이디어 : 임베딩 된 중심 단어와 주변 단어 벡터의 내적이 전체 코퍼스에서의 동시 등장 확률이 되도록 만드는 것

8.5.4 학습 기반 임베딩

- Seq2Seq 모델 (Sequence-to-Sequence)
  - 입력된 시퀀스로부터 다른 도메인의 시퀀스를 출력하는 다양한 분야에서 사용되는 모델